ServiX

Gestión de clientes, servicios y atención con IA

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

SRS y Documento Académico de ServiX

Especificación de requisitos, marcos teórico, técnico y legal, metodología, resultados y viabilidad del proyecto integrador

Proyecto	ServiX — Gestión de clientes, servicios y atención con IA
Código del documento	DOC-01
Versión	1.0
Fecha	01/11/2026
Estándar de referencia	ISO/IEC/IEEE 29148:2018 (correspondencia con IEEE 830-1998)
Institución	Universidad de San Buenaventura
Asignatura	Optativa II
Docente	Ing. Jairo Armando Salcedo Aranda

Equipo de trabajo

Nicolas Quiroz Gomez	Product Owner
Juan Sebastian Garzon Suarez	Scrum Master
Juan Esteban Olaya Muñoz	Developer

Control del documento

Información general

Título	SRS y Documento Académico de ServiX
Código	DOC-01
Versión	1.0
Fecha de emisión	01/11/2026
Estado	Versión 1.0 emitida; aprobaciones pendientes
Proyecto	ServiX
Autor	Equipo ServiX
Clasificación	Uso académico; sin información confidencial

Historial de revisiones

Versión	Fecha	Descripción del cambio	Autor
0.9	01/11/2026	Borrador inicial en formato plano (archivado como respaldo v1).	Equipo ServiX
1.0	01/11/2026	Rediseño completo: portada, control de versiones, numeración, figuras y tablas rotuladas, mayor profundidad.	Equipo ServiX

Aprobaciones

Rol	Nombre	Fecha	Firma / estado
Elaboró	Juan Esteban Olaya Muñoz (Developer)	Pendiente	Pendiente
Revisó	Juan Sebastian Garzon Suarez (Scrum Master)	Pendiente	Pendiente
Aprobó	Nicolas Quiroz Gomez (Product Owner)	Pendiente	Pendiente

Las aprobaciones se encuentran **pendientes**: no se registran firmas ni fechas de aprobación hasta que los responsables las emitan.

Tabla de contenido

Resumen ejecutivo **9**
1 Introducción **10**
1.1 Propósito 10
1.2 Alcance 10
1.3 Definiciones, acrónimos y abreviaturas 11
1.4 Referencias 12
1.5 Visión general del documento 12
2 Planteamiento del problema y justificación **13**
2.1 Contexto 14
2.2 Planteamiento del problema 14
2.3 Árbol de problemas 15
2.4 Justificación 15
3 Objetivos **16**
3.1 Objetivo general 16
3.2 Objetivos específicos 16
4 Alcance, delimitación y limitaciones **17**
4.1 Alcance 17
4.2 Delimitación 17
4.3 Limitaciones 18
5 Descripción general del producto **19**
5.1 Visión del producto 19
5.2 Perspectiva del producto 19
5.3 Funciones del producto 20
5.4 Usuarios y roles 20
5.5 Entorno de operación 21
5.6 Restricciones 21
5.7 Suposiciones y dependencias 22
5.8 Módulos del producto 22
5.9 Flujo principal de uso 23
5.10 Diferenciadores 23
5.11 Capturas de la aplicación 24
6 Marco teórico **26**
6.1 Sistemas de información y gestión de servicios 26
6.2 Desarrollo móvil nativo en Android 26
6.3 Arquitectura por capas, MVVM y SOLID 26
6.4 Bases de datos NoSQL orientadas a documentos 26
6.5 Inteligencia artificial generativa y modelos de lenguaje 27
6.6 Scrum 27
6.7 DevOps e integración continua 27
6.8 Pruebas de software 27
7 Marco técnico y tecnológico **27**
7.1 Stack tecnológico 27
7.2 Alternativas consideradas 28

7.3 Arquitectura . . . 29
7.4 Seguridad técnica . . . 30
7.5 CI/CD . . . 31
8 Marco conceptual . . . 31
9 Marco legal y normativo . . . 32
9.1 Alcance y advertencia . . . 33
9.2 Protección de datos personales . . . 33
9.3 Delitos informáticos . . . 33
9.4 Propiedad intelectual y licencias . . . 33
9.5 Mensajes de datos y comercio electrónico . . . 34
9.6 Términos de servicio de terceros . . . 34
9.7 Uso responsable de la inteligencia artificial . . . 34
9.8 Estándares técnicos . . . 34
9.9 Obligaciones, tratamiento actual y brechas . . . 35
10 Consideraciones éticas . . . 35
11 Metodología . . . 36
11.1 Marco de trabajo . . . 36
11.2 Sprints . . . 36
11.3 Backlog . . . 37
11.4 Definición de terminado . . . 37
11.5 Herramientas y flujo de ramas . . . 37
11.6 Gestión de riesgos . . . 38
12 Requisitos específicos . . . 39
12.1 Requisitos funcionales . . . 39
12.2 Requisitos no funcionales . . . 46
12.3 Interfaces externas . . . 49
12.4 Requisitos de seguridad . . . 50
12.5 Modelo de datos . . . 51
12.6 Casos de uso . . . 52
12.7 Matriz de trazabilidad resumida . . . 53
12.8 Criterios de aceptación y verificación . . . 54
13 Resultados verificables y limitaciones . . . 54
13.1 Resultados obtenidos . . . 55
13.2 Estado de los requisitos . . . 55
13.3 Limitaciones honestas . . . 55
14 Viabilidad . . . 56
14.1 Viabilidad técnica . . . 56
14.2 Viabilidad operativa . . . 56
14.3 Viabilidad económica . . . 56
15 Conclusiones, recomendaciones y trabajo futuro . . . 57
15.1 Conclusiones . . . 57
15.2 Recomendaciones . . . 57
15.3 Trabajo futuro . . . 57
Referencias . . . 58
Anexo A. Matriz de trazabilidad de historias de usuario . . . 60

Anexo B. Registro de riesgos **62**
Anexo C. Hallazgos de la auditoría técnica (H-01 a H-21) **64**
Anexo D. Enlaces y recursos **66**

Lista de figuras

Figura 1. Árbol de causas y efectos del problema central 15
Figura 2. Diagrama de contexto de ServiX y sus sistemas externos 19
Figura 3. Acceso, registro, Portal del cliente y chat del cliente 24
Figura 4. Inicio, notificaciones, lista de clientes y detalle de cliente 24
Figura 5. Lista de servicios, detalle de servicio, historial y perfil 25
Figura 6. Formularios de alta, gestión de solicitudes de atención y chat del administrador 25
Figura 7. Arquitectura por capas de ServiX 29
Figura 8. Componentes principales y sus dependencias 30
Figura 9. Flujo de integración continua en Azure Pipelines 31
Figura 10. Modelo de datos de ServiX en Cloud Firestore 51
Figura 11. Diagrama de casos de uso de ServiX 52

Lista de tablas

Tabla 1. ServiX en una página . . . 9
Tabla 2. Definiciones y acrónimos usados en el documento . . . 11
Tabla 3. Normas y estándares de referencia . . . 12
Tabla 4. Documentos y artefactos del proyecto relacionados . . . 12
Tabla 5. Convenciones de identificación, prioridad y verificación . . . 13
Tabla 6. Correspondencia entre IEEE 830 y la estructura adoptada (ISO/IEC/IEEE 29148) . . . 13
Tabla 7. Objetivos específicos, indicadores y evidencias . . . 16
Tabla 8. Delimitación del proyecto . . . 17
Tabla 9. Limitaciones del proyecto . . . 18
Tabla 10. Resumen de funciones por módulo . . . 20
Tabla 11. Características de los usuarios . . . 20
Tabla 12. Matriz de permisos por rol . . . 20
Tabla 13. Restricciones de diseño e implementación . . . 21
Tabla 14. Suposiciones y dependencias . . . 22
Tabla 15. Stack tecnológico y versiones . . . 28
Tabla 16. Alternativas consideradas y razón de elección . . . 28
Tabla 17. Glosario de conceptos . . . 31
Tabla 18. Licencias de las dependencias utilizadas (a verificar en cada proyecto) . . . 33
Tabla 19. Estándares técnicos de referencia . . . 34
Tabla 20. Obligación, cómo la aborda ServiX hoy y brecha o acción pendiente . . . 35
Tabla 21. Consideraciones éticas y su aplicación . . . 36
Tabla 22. Roles Scrum del equipo . . . 36
Tabla 23. Sprints (fechas provisionales) . . . 36
Tabla 24. Elementos principales del backlog . . . 37
Tabla 25. Herramientas del proyecto . . . 37
Tabla 26. Fichas de requisitos funcionales: autenticación y roles (RF-001 a RF-005) . . . 39
Tabla 27. Fichas de requisitos funcionales: gestión de clientes (RF-006 a RF-009) . . . 40
Tabla 28. Fichas de requisitos funcionales: gestión de servicios (RF-010 a RF-011) . . . 42
Tabla 29. Fichas de requisitos funcionales: historial, notificaciones y búsqueda (RF-012 a RF-014) . . . 42
Tabla 30. Fichas de requisitos funcionales: panel de inicio, perfil y configuración (RF-015 a RF-023) . . . 43
Tabla 31. Fichas de requisitos funcionales: asistente de ia y atención al cliente (RF-018 a RF-021) . . . 44
Tabla 32. Ficha del requisito funcional RF-022: integración con make . . . 46
Tabla 33. Requisitos no funcionales de seguridad . . . 46
Tabla 34. Requisitos no funcionales de rendimiento . . . 47
Tabla 35. Requisitos no funcionales de usabilidad . . . 47
Tabla 36. Requisitos no funcionales de mantenibilidad . . . 48
Tabla 37. Requisitos no funcionales de portabilidad . . . 48
Tabla 38. Requisitos no funcionales de disponibilidad . . . 48
Tabla 39. Destinos de navegación de la aplicación . . . 49
Tabla 40. Interfaces de software . . . 49
Tabla 41. Requisitos de seguridad . . . 50
Tabla 42. Colecciones de Firestore . . . 51
Tabla 43. Descripción de los casos de uso . . . 52

Tabla 44. Trazabilidad resumida por módulo: requisitos, historias y casos de prueba 53
Tabla 45. Métodos de verificación aplicados 54
Tabla 46. Resultados verificables del proyecto 55
Tabla 47. Estado de verificación de los requisitos funcionales 55
Tabla 48. Costos y recursos del proyecto 56
Tabla 49. Recomendaciones priorizadas 57
Tabla 50. Historias de usuario, requisitos y elementos del backlog 60
Tabla 51. Riesgos identificados 62
Tabla 52. Hallazgos de la auditoría y su estado 64
Tabla 53. Enlaces del proyecto 66

Resumen ejecutivo

ServiX es una aplicación móvil nativa para Android que ayuda a pequeños negocios de servicios técnicos para el hogar a organizar su cartera de clientes, sus órdenes de servicio y la atención de reclamos. Combina una gestión sencilla de clientes y servicios con un asistente de inteligencia artificial que responde consultas y, cuando el caso lo exige (una queja, una cancelación o un problema), lo escala a una persona mediante una solicitud de atención con prioridad y radicado.

Este documento es la **especificación de requisitos de software (SRS)** y, a la vez, el **documento académico principal** del proyecto integrador de Optativa II. Por ello integra, además de los requisitos, el planteamiento del problema, los objetivos, los marcos teórico, técnico, conceptual y legal, la metodología, los resultados verificables, la viabilidad y las conclusiones. Todo lo que se afirma sobre el sistema proviene del código fuente, de las reglas de seguridad desplegadas, del pipeline de integración continua y de los resultados de pruebas realmente ejecutados.

Tabla 1. ServiX en una página

Aspecto	Síntesis
Tipo de producto	Aplicación Android nativa (Kotlin, Jetpack Compose, Material 3), sin servidor propio.
Usuarios	Cliente y Administrador, con rol almacenado en la cuenta y reglas de seguridad en Firestore.
Módulos	Autenticación y roles, clientes, servicios, historial y notificaciones, panel y configuración, asistente de IA y atención, integración opcional con Make.
Servicios en la nube	Firebase Authentication, Cloud Firestore y Firebase AI Logic (Gemini 2.5 Flash).
Gestión del proyecto	Scrum en Azure DevOps (3 Sprints provisionales, 12 PBI, 27 tareas, 6 bugs) y pipeline ServiX-CI.
Requisitos	23 funcionales y 19 no funcionales, 11 de seguridad, 9 casos de uso.
Verificación	29 de 29 pruebas automatizadas aprobadas (local 31/10/2026; Azure Pipelines 01/11/2026); APK de depuración publicado como artefacto.
No ejecutado	Pruebas en dispositivo, con Firebase y Gemini reales, con Make real, Android Lint, pruebas instrumentadas y de reglas.
Brechas principales	Autoasignación del rol Administrador en el registro; sin autorización ni política de tratamiento de datos personales; sin App Check; sin archivo LICENSE.

> **Advertencia. Naturaleza del documento**
> El marco legal de este documento es un **análisis académico**, no asesoría jurídica. Antes de operar ServiX con datos reales de personas deben verificarse las normas vigentes en fuentes oficiales y consultarse a un profesional del derecho.

1 Introducción

1.1 Propósito

Este documento especifica los requisitos de la aplicación móvil **ServiX**, una solución Android para la gestión de clientes, servicios técnicos y atención al cliente asistida por inteligencia artificial. Describe qué debe hacer el sistema y bajo qué restricciones, de modo que el equipo de desarrollo, el Product Owner y el docente de la asignatura compartan una misma referencia verificable.

La especificación se redactó a partir del código fuente real del repositorio, de las reglas de seguridad de Firestore, de la configuración del pipeline de integración continua y de los resultados de las pruebas ejecutadas. Cada requisito funcional indica en qué archivos del código se origina y cuál es su estado de verificación, de manera que ninguna afirmación del documento depende de supuestos no comprobados.

Audiencia prevista: (1) el equipo Scrum de ServiX, que usa el documento como base de planificación y de pruebas; (2) el docente de Optativa II, que lo evalúa como parte del proyecto integrador; y (3) cualquier persona que deba mantener o ampliar la aplicación en el futuro.

1.2 Alcance

ServiX es una aplicación Android nativa (Kotlin y Jetpack Compose) con persistencia en Cloud Firestore, autenticación con Firebase Authentication y un asistente conversacional basado en Gemini a través de Firebase AI Logic. Atiende a dos tipos de usuario: el **cliente**, que solicita servicios y consulta su estado, y el **administrador**, que gestiona la cartera de clientes, las órdenes de servicio y las solicitudes de atención.

1.2.1 Lo que el sistema sí hace

- Registra usuarios, autentica con correo y contraseña y aplica el rol almacenado en la cuenta (Cliente o Administrador).
- Gestiona clientes y servicios (alta, edición, duplicado, eliminación con opción de deshacer, favoritos, estados y prioridades).
- Calcula segmentos de cliente (VIP, Frecuente, Nuevo, Pendiente, Inactivo) y estadísticas del negocio.
- Registra un historial de actividades y genera notificaciones internas.
- Ofrece un asistente de chat con IA que responde con contexto del usuario y clasifica la intención del mensaje.
- Escala quejas, cancelaciones y problemas a una solicitud de atención que el administrador toma, responde y resuelve.
- Envía eventos de servicio (creado, actualizado, completado) a un webhook de Make cuando este se configura.
- Se compila, prueba y empaqueta automáticamente en Azure Pipelines, que publica el APK de depuración como artefacto.

1.2.2 Lo que el sistema no hace

- No gestiona pagos, facturación electrónica ni pasarelas de cobro; el precio de un servicio es un dato informativo.
- No asigna técnicos ni programa rutas; no existe un rol de técnico independiente.
- No implementa las interfaces `RetellAIRepository` ni `N8nWebhookRepository`: están declaradas como contratos futuros sin código asociado.
- No ofrece una versión web ni para iOS; el único cliente es la aplicación Android.
- No envía notificaciones push del sistema operativo; las notificaciones son internas y se leen en la propia aplicación.
- No modifica roles desde la interfaz: el rol se fija en el registro y las reglas de Firestore impiden cambiarlo después.
- No ha sido verificado en dispositivo físico ni emulador, ni contra los servicios reales de Firebase y Gemini (véase la sección 12.8).

1.3 Definiciones, acrónimos y abreviaturas

Tabla 2. Definiciones y acrónimos usados en el documento

Término	Definición
ADR	Architecture Decision Record: registro de una decisión arquitectónica con su contexto y consecuencias.
APK	Android Package: archivo instalable de una aplicación Android.
Cliente	Usuario final que solicita servicios y consulta el estado de sus órdenes (rol `CLIENTE`).
Administrador	Usuario que gestiona clientes, servicios y solicitudes de atención (rol `ADMINISTRADOR`; en el código también se aceptan los valores heredados `ADMIN`, `VENDEDOR` y `ADMIN_VENDEDOR`).
CI/CD	Integración continua y entrega continua; en el proyecto solo se implementa la parte de integración continua.
Firestore	Cloud Firestore: base de datos de documentos en la nube de Google (Firebase).
Firebase AI Logic	Servicio de Firebase que expone modelos Gemini a las aplicaciones sin incluir la clave de API en el cliente.
Gemini	Familia de modelos de lenguaje de Google; ServiX usa `gemini-2.5-flash`.
Make	Plataforma de automatización de flujos que recibe eventos mediante webhooks (integración opcional).
MVVM	Model-View-ViewModel: patrón de presentación usado por la aplicación.
PBI	Product Backlog Item: elemento del backlog de producto en Azure Boards.
Radicado	Código corto de 8 caracteres que identifica una solicitud de atención creada desde el chat.
Robolectric	Framework que ejecuta pruebas de Android en la JVM sin dispositivo.
RF / RNF	Requisito funcional / requisito no funcional.
Solicitud de atención	Registro en la colección `solicitudes_atencion` que representa un caso escalado a una persona.
SRS	Software Requirements Specification.

1.4 Referencias

1.4.1 Referencias normativas

Tabla 3. Normas y estándares de referencia

Referencia	Uso en este documento
ISO/IEC/IEEE 29148:2018 — Requirements engineering	Estructura del documento, atributos de calidad de los requisitos y trazabilidad.
IEEE Std 830-1998 — Recommended Practice for SRS	Correspondencia de secciones (véase la Tabla 3) y criterios de buena especificación.
ISO/IEC 25010:2011 — Modelo de calidad del producto	Clasificación de los requisitos no funcionales por categoría.
ISO/IEC/IEEE 29119-3:2021 — Documentación de pruebas	Vínculo con el plan y el informe de pruebas del proyecto.

1.4.2 Referencias del proyecto

Tabla 4. Documentos y artefactos del proyecto relacionados

Código	Documento o artefacto	Relación con el SRS
DOC-02	Arquitectura y Diseño (IEEE 1016 / 42010)	Realiza los requisitos mediante la arquitectura en capas.
DOC-03	Plan de Pruebas (IEEE 29119-3)	Define cómo se verifica cada requisito.
DOC-04	Informe de Pruebas	Resultados reales de las 29 pruebas automatizadas.
DOC-07	Documentación de Firebase	Detalle del modelo de datos y de las reglas de seguridad.
DOC-08	Documentación de IA	Detalle del asistente conversacional.
—	Guía Final – Azure DevOps (Optativa II)	Requisitos académicos del proyecto integrador.
—	Matrices de trazabilidad (Excel)	Detalle completo de trazabilidad requisito, historia, caso y evidencia.

1.5 Visión general del documento

El documento sigue el orden de un trabajo académico de ingeniería de software. Las secciones 2 a 4 plantean el problema, los objetivos y el alcance; la sección 5 describe el producto; las secciones 6 a 9 desarrollan los marcos teórico, técnico, conceptual y legal; la sección 10 trata las consideraciones éticas y la 11 la metodología; la sección 12 contiene los requisitos específicos (funcionales con ficha por módulo, no funcionales, interfaces, seguridad, datos, casos de uso, trazabilidad y verificación); y las secciones 13 a 15 presentan los resultados, la viabilidad y las conclusiones. Siguen las referencias y los anexos.

1.5.1 Convenciones

Tabla 5. Convenciones de identificación, prioridad y verificación

Elemento	Convención
Identificadores	`RF-nnn` requisito funcional; `RNF-nnn` no funcional; `SEC-nn` seguridad; `CU-nn` caso de uso; `HU-nnn` historia de usuario; `CP-nnn` caso de prueba.
Prioridad	**Alta** (imprescindible para el objetivo del producto), **Media** (importante, con alternativa temporal), **Baja** (deseable).
Estado: Cumplido	El comportamiento está implementado y existe verificación suficiente (prueba automatizada aprobada o inspección completa del código).
Estado: Parcialmente cumplido	Implementado, pero con verificación incompleta o con una limitación conocida.
Estado: No verificable	Implementado en código, pero su comprobación exige dispositivo, emulador o servicios externos que aún no se han ejecutado.
Estado: No cumplido	No implementado.

1.5.2 Correspondencia con IEEE 830

Tabla 6. Correspondencia entre IEEE 830 y la estructura adoptada (ISO/IEC/IEEE 29148)

IEEE 830	Este documento
1. Introducción (propósito, alcance, definiciones, referencias, visión general)	Sección 1
2. Descripción general (perspectiva, funciones, usuarios, restricciones, suposiciones)	Sección 5
3.1 Interfaces externas	Sección 12.3
3.2 Funciones (requisitos funcionales)	Sección 12.1
3.3 Requisitos de rendimiento	Sección 12.2
3.4 Restricciones de diseño	Sección 5.6
3.5 Atributos del sistema (seguridad, mantenibilidad, portabilidad, disponibilidad)	Secciones 12.2 y 12.4
3.6 Otros requisitos (datos, casos de uso)	Secciones 12.5 y 12.6
Apéndices	Anexos A a D

2 Planteamiento del problema y justificación

2.1 Contexto

En Colombia, como en muchos países, una parte importante de los servicios técnicos para el hogar y para pequeños negocios (climatización, redes, mantenimiento eléctrico o de plomería, instalación de equipos) la prestan técnicos independientes y microempresas. Estos negocios suelen gestionar la información de sus clientes y de sus órdenes de trabajo con medios informales: cuadernos, hojas de cálculo, notas del teléfono y conversaciones de mensajería. Esa forma de trabajo es flexible y no requiere inversión, pero se vuelve frágil cuando crece el número de clientes o cuando varias personas intervienen en la atención.

Este contexto es un planteamiento del equipo basado en la observación del dominio y en el diseño del producto; no se apoya en estadísticas ni en un estudio de campo, y por eso no se citan cifras de mercado.

2.2 Planteamiento del problema

La gestión desarticulada de clientes, servicios y atención genera, en este tipo de negocios, órdenes que se olvidan o se retrasan, información de contacto dispersa, reclamos que llegan por canales distintos y sin un responsable claro, y poca capacidad para saber qué trabajos están pendientes, cuáles se completaron y cuánto se facturó. Las soluciones empresariales de gestión de servicios existen, pero con frecuencia resultan complejas o costosas para un técnico independiente o una microempresa.

> **Definición. Pregunta de investigación**
> ¿Cómo puede una aplicación móvil nativa, de bajo costo y apoyada en servicios en la nube, ayudar a un pequeño negocio de servicios técnicos a gestionar sus clientes y órdenes y a atender de forma ordenada las consultas y reclamos, escalando a una persona cuando la inteligencia artificial no es suficiente?

2.3 Árbol de problemas

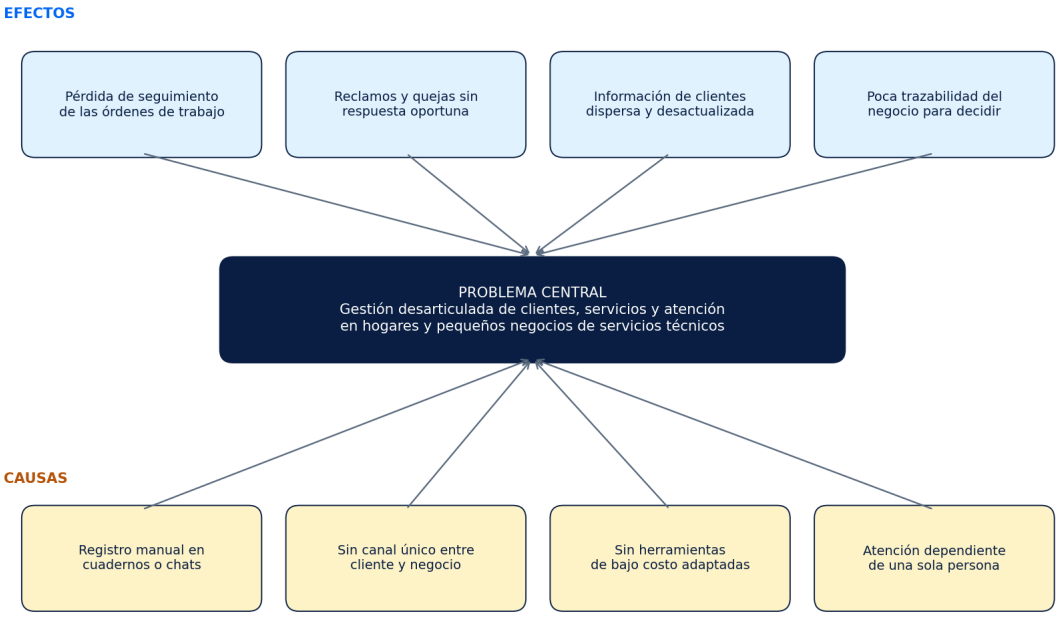


Figura 1. Árbol de causas y efectos del problema central

Fuente: Elaboración propia (Equipo ServiX); planteamiento cualitativo, sin estadísticas.

Las causas descritas en la parte inferior del árbol alimentan el problema central: la información vive en medios informales, no existe un canal único entre el cliente y el negocio, las herramientas de bajo costo no se adaptan al dominio y la atención depende de una sola persona. Como consecuencia aparecen los efectos de la parte superior: seguimiento deficiente de las órdenes, reclamos sin respuesta oportuna, información desactualizada y poca visibilidad del negocio para decidir.

2.4 Justificación

2.4.1 Justificación académica

ServiX es el proyecto integrador de la asignatura Optativa II del programa de Tecnología en Desarrollo de Software. Permite aplicar de forma conjunta la ingeniería de requisitos, el diseño de software por capas, el desarrollo móvil nativo, el uso de servicios en la nube, la integración de inteligencia artificial, las pruebas automatizadas y las prácticas de Scrum y de integración continua con Azure DevOps, tal como exige la guía de la asignatura.

2.4.2 Justificación técnica

El problema se presta a una solución móvil: los técnicos trabajan fuera de una oficina y el teléfono es su herramienta principal. Firebase ofrece autenticación, base de datos en tiempo real y acceso a modelos de lenguaje sin que el equipo mantenga un servidor, lo que reduce la complejidad operativa. La arquitectura por capas permite probar la lógica en la JVM y sustituir proveedores (por ejemplo, el de IA) con poco impacto.

2.4.3 Justificación social

Una herramienta sencilla puede ayudar a microempresarios y técnicos independientes a ofrecer una atención más ordenada y transparente a sus clientes, que a su vez pueden conocer el estado de sus solicitudes y contar con un canal claro para reclamos. El escalamiento a una persona reconoce que la IA no sustituye la atención humana en situaciones sensibles.

2.4.4 Justificación económica

El proyecto se construye con herramientas gratuitas o de uso académico (Android Studio, Azure DevOps, Firebase y un repositorio público), por lo que el costo directo de desarrollo es bajo. Para un negocio pequeño, una aplicación propia basada en servicios gestionados evita licencias por usuario y permite empezar sin infraestructura. Los costos de operación dependen del uso de Firebase y de Gemini y se analizan en la sección de viabilidad.

3 Objetivos

3.1 Objetivo general

Desarrollar y documentar una aplicación móvil nativa para Android que permita gestionar clientes, servicios técnicos y solicitudes de atención con apoyo de un asistente de inteligencia artificial, aplicando prácticas de ingeniería de software verificables (requisitos, arquitectura, pruebas, Scrum y integración continua) durante el periodo de la asignatura Optativa II.

3.2 Objetivos específicos

Cada objetivo se formula con un verbo medible y se acompaña de un indicador y de una evidencia que puede comprobarse en el repositorio o en la documentación.

Tabla 7. Objetivos específicos, indicadores y evidencias

ID	Objetivo específico	Indicador	Evidencia verificable	Estado
OE-1	Implementar la autenticación con roles Cliente y Administrador.	RF-001 a RF-005 implementados; reglas con rol inmutable.	Código de autenticación, `firestore.rules`, CP-014 a CP-017 aprobados.	Parcial: login por rol sin probar en dispositivo
OE-2	Construir la gestión de clientes y servicios con estados, prioridades y restauración.	RF-006 a RF-011 implementados.	Pruebas CP-005 a CP-012 aprobadas.	Cumplido (datos simulados)
OE-3	Integrar un asistente de IA con escalamiento a atención humana.	Flujo de chat, clasificación de intención y solicitudes de atención implementados.	`GeminiAiChatRepository`, CP-018 a CP-021 aprobados.	Parcial: Gemini real no verificado

ID	Objetivo específico	Indicador	Evidencia verificable	Estado
OE-4	Proteger los datos con reglas de mínimo privilegio y sin secretos en el repositorio.	Reglas desplegadas; sin claves en el código.	`firestore.rules`, revisión del repositorio (H-01, H-02).	Parcial: sin pruebas de reglas
OE-5	Verificar el sistema con pruebas automatizadas ejecutables en integración continua.	29 pruebas; pipeline en verde.	Informe JUnit; run 20261101.3 de Azure Pipelines.	Cumplido
OE-6	Gestionar el proyecto con Scrum en Azure DevOps.	Organización, proyecto, 3 Sprints, backlog, tablero, dashboard, repositorio y PR.	Capturas reales (DOC-09).	Cumplido (fechas provisionales)
OE-7	Documentar el sistema con estándares de ingeniería de software.	Documentos DOC-01 a DOC-10 en versión 1.0.	Carpeta `DOCUMENTACION_SERVIX`.	Cumplido; aprobaciones pendientes
OE-8	Identificar y registrar riesgos, brechas y obligaciones legales.	Registro de riesgos y tabla del marco legal.	Anexos B y C y sección 9.	Cumplido (brechas abiertas)

4 Alcance, delimitación y limitaciones

4.1 Alcance

El alcance funcional del producto, lo que el sistema sí hace y lo que no hace, se describe en la sección 1.2. Esta sección precisa los límites del proyecto como trabajo académico.

4.2 Delimitación

Tabla 8. Delimitación del proyecto

Dimensión	Delimitación
Temporal	Tres Sprints de dos semanas (2 de noviembre a 11 de diciembre de 2026, fechas provisionales) dentro del periodo de Optativa II.
Funcional	Gestión de clientes, servicios, historial, notificaciones, chat con IA, atención y portal del cliente; sin pagos, rutas ni asignación de técnicos.
Tecnológica	Android nativo (API 26 a 36), Firebase (Authentication, Firestore, AI Logic) y Make opcional.
Poblacional	Pequeños negocios de servicios técnicos y sus clientes; no se realizó un estudio con usuarios reales.
Geográfica	Contexto colombiano; la interfaz y los documentos están en español.
Entrega	APK de depuración; no se publica en una tienda de aplicaciones.

4.3 Limitaciones

Tabla 9. Limitaciones del proyecto

Limitación	Efecto	Origen
Equipo de desarrollo con 4 GB de RAM y sin virtualización.	Compilaciones de más de 20 minutos; sin emulador.	Recursos del equipo.
Ausencia de pruebas en dispositivo y con servicios reales.	El comportamiento extremo a extremo no está verificado.	Falta de dispositivo disponible durante el desarrollo.
Fechas de los Sprints provisionales.	El calendario puede cambiar.	Pendiente de confirmar con el calendario académico.
Dos integrantes con invitación pendiente en Azure DevOps.	No pueden actualizar el tablero aún.	Gestión de accesos.
Sin estudio de mercado ni pruebas de usabilidad.	La justificación es cualitativa.	Alcance de la asignatura.
Dependencia de servicios de terceros (Firebase y Gemini).	Cuotas, costos y disponibilidad ajenos al equipo.	Decisión de arquitectura.
Sin tratamiento formal de datos personales.	No se debería operar con datos reales sin las medidas del marco legal.	Brecha identificada (sección 9).

5 Descripción general del producto

5.1 Visión del producto

ServiX aspira a ser la herramienta sencilla que un técnico independiente o una microempresa de servicios abre cada mañana para saber qué trabajos tiene, a quién debe atender y qué reclamos están pendientes, y que a su vez da a sus clientes un canal claro para solicitar servicios y recibir respuestas. La inteligencia artificial se usa para resolver las consultas repetitivas y las personas se reservan para los casos que requieren criterio.

5.2 Perspectiva del producto

ServiX es una aplicación autónoma de un solo módulo Android (app) que se apoya en servicios gestionados de Firebase. No existe un servidor propio: la lógica de negocio se ejecuta en el dispositivo y la persistencia, la autenticación y la IA se delegan en Firebase. La integración con Make es opcional y unidireccional (la aplicación envía eventos; no recibe respuestas de negocio).

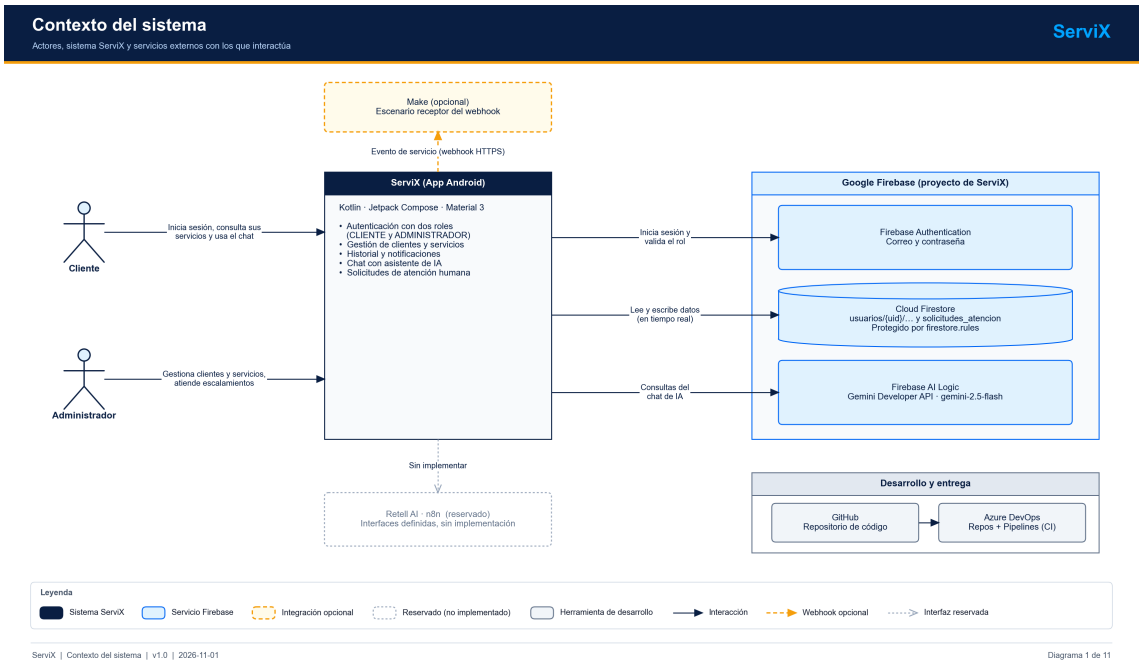


Figura 2. Diagrama de contexto de ServiX y sus sistemas externos

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

El diagrama muestra a los dos actores humanos, la aplicación y cuatro sistemas externos: Firebase Authentication, Cloud Firestore, Firebase AI Logic (Gemini) y el webhook de Make. Azure DevOps interviene únicamente en el ciclo de desarrollo (repositorio, tablero y pipeline), no en la ejecución de la aplicación.

5.3 Funciones del producto

Tabla 10. Resumen de funciones por módulo

Módulo	Funciones principales	Requisitos
Autenticación y roles	Registro, inicio y cierre de sesión, recuperación de contraseña, navegación según rol.	RF-001 a RF-005
Clientes	Alta, edición, eliminación con restauración, favoritos y segmentación automática.	RF-006 a RF-009
Servicios	Alta, edición, duplicado, eliminación, estados, prioridades y progreso.	RF-010, RF-011
Seguimiento	Historial de actividades, notificaciones internas y búsqueda global.	RF-012 a RF-014
Panel y configuración	Estadísticas, metas semanales, perfil del negocio, ajustes, tutorial y restablecimiento.	RF-015 a RF-017, RF-023
Asistente y atención	Chat con IA, clasificación de intención, escalamiento, gestión de solicitudes y portal del cliente.	RF-018 a RF-021
Integración	Eventos de servicio hacia Make y prueba de conexión.	RF-022

5.4 Usuarios y roles

La aplicación distingue dos roles, definidos en la enumeración `UserRole`. El rol se almacena en el campo `role` del documento `usuarios/{uid}`. Si el rol no puede leerse, el sistema aplica el principio de mínimo privilegio y trata la cuenta como `CLIENTE`.

Tabla 11. Características de los usuarios

Rol	Descripción	Experiencia esperada	Pantallas principales
Cliente	Persona que contrata servicios técnicos y consulta su avance.	Uso básico de aplicaciones móviles.	Portal del cliente y Chat IA.
Administrador	Responsable del negocio que gestiona clientes, servicios y casos de atención.	Uso habitual de aplicaciones de gestión.	Inicio, Clientes, Servicios, Historial, Chat IA, Perfil.
Anónimo	Persona sin sesión iniciada.	—	Pantalla de autenticación.

Tabla 12. Matriz de permisos por rol

Capacidad	Anónimo	Cliente	Administrador	Mecanismo de control
Registrarse e iniciar sesión	Sí	—	—	Firebase Authentication
Ver el Portal del cliente	No	Sí	No (navega al panel)	Navegación por rol en `MainActivity`
Gestionar clientes y servicios	No	No (la interfaz no lo ofrece)	Sí	Interfaz; reglas: dueño del árbol o administrador
Ver historial, notificaciones y metas	No	No	Sí	Interfaz; reglas por usuario
Conversar con el asistente de IA	No	Sí	Sí	Sesión autenticada

Capacidad	Anónimo	Cliente	Administrador	Mecanismo de control
Crear una solicitud de atención	No	Sí (solo a su nombre)	Sí (solo a su nombre)	Regla: `clienteId == request.auth.uid`
Leer solicitudes de atención	No	Solo las propias	Todas	Regla de lectura en `solicitudes_atencion`
Tomar, responder o resolver solicitudes	No	No	Sí	Regla: `update, delete` solo `isAdmin()`
Cambiar el rol de una cuenta	No	No	Solo mediante regla de administrador	Regla: el rol es inmutable para el dueño

Advertencia. Autoasignación de rol en el registro
La pantalla de registro permite elegir el rol Administrador y las reglas de Firestore aceptan `ADMINISTRADOR` en la creación del perfil. Esto permite que cualquier persona obtenga privilegios de administrador. Es un riesgo conocido y abierto (hallazgo H-15, Bug 45, riesgo R-01); la solución prevista es asignar el rol de administrador desde un entorno de confianza (Admin SDK o Cloud Function).

5.5 Entorno de operación

- Dispositivos Android 8.0 (API 26) o superior; la aplicación apunta a la API 36.
- Conectividad a Internet para autenticación, sincronización con Firestore y asistente de IA.
- Proyecto Firebase con Authentication (correo y contraseña), Firestore y Firebase AI Logic habilitados.
- Para el desarrollo: JDK 21, Android SDK 36.1, Gradle mediante el wrapper del repositorio (equipo de referencia: Windows 10, 4 GB de RAM).

5.6 Restricciones

Tabla 13. Restricciones de diseño e implementación

ID	Restricción	Origen
RES-01	El proyecto debe desarrollarse en Kotlin con Jetpack Compose y Material 3.	Decisión técnica del equipo; perfil del curso.
RES-02	Persistencia y autenticación en Firebase (Firestore y Authentication).	Decisión del proyecto; no hay backend propio.
RES-03	No se pueden empaquetar claves de IA en el APK; se usa Firebase AI Logic.	Hallazgo de auditoría H-02.
RES-04	Los secretos (URL del webhook, claves de firma) no se versionan; se leen de `local.properties` o variables de entorno.	Práctica de seguridad del equipo.
RES-05	El proyecto debe gestionarse con Azure DevOps (Scrum, repositorio, pipeline y dashboard).	Guía Final – Azure DevOps de Optativa II.
RES-06	El pipeline de CI usa un agente alojado `ubuntu-latest` con JDK 21 (exigido por Robolectric con SDK 36).	Falla real del run 2 del pipeline.
RES-07	El equipo de desarrollo de referencia dispone de 4 GB de RAM y no admite virtualización; las compilaciones son lentas.	Entorno real del equipo.

5.7 Suposiciones y dependencias

Tabla 14. Suposiciones y dependencias

ID	Tipo	Descripción	Impacto si falla
SUP-01	Suposición	El proyecto Firebase `servix-453fb` permanece activo con las reglas de seguridad desplegadas.	La aplicación no puede persistir ni autenticar.
SUP-02	Suposición	El usuario dispone de conexión estable al usar el asistente de IA.	El chat devuelve un mensaje de error.
DEP-01	Dependencia	Firebase Authentication y Cloud Firestore (BoM de Firebase 34.17.0).	Sin ellos, el sistema recurre a repositorios simulados (véase ADR-05 del documento de arquitectura).
DEP-02	Dependencia	Firebase AI Logic con Gemini Developer API y modelo `gemini-2.5-flash`.	Chat sin respuestas de IA; el escalamiento por palabras clave sigue disponible.
DEP-03	Dependencia opcional	Webhook de Make configurado en `local.properties` o en `MAKE_WEBHOOK_URL`.	Los eventos se omiten (con aviso en el registro).
DEP-04	Dependencia de desarrollo	Azure DevOps (organización Serivx, proyecto ServiX) y Azure Pipelines.	Sin CI no hay verificación automática de cada cambio.

5.8 Módulos del producto

5.8.1 Autenticación y roles

Permite registrarse, iniciar y cerrar sesión y recuperar la contraseña. El rol (Cliente o Administrador) se guarda en el perfil del usuario y determina qué pantallas ve. El inicio de sesión rechaza el acceso si el rol elegido no coincide con el almacenado, y si el rol no puede leerse se aplica el mínimo privilegio.

5.8.2 Clientes

El administrador registra clientes con sus datos de contacto y notas, los edita, los marca como favoritos y los elimina con posibilidad de deshacer. El sistema calcula segmentos (VIP, Frecuente, Nuevo, Pendiente, Inactivo) a partir de los servicios y la antigüedad.

5.8.3 Servicios

Cada orden de servicio tiene categoría, precio informativo, fechas, prioridad y estado (Pendiente, En Proceso, Completado, Cancelado). Puede duplicarse, editarse y eliminarse, y su avance se muestra en un indicador de progreso.

5.8.4 Historial y notificaciones

Cada operación relevante deja una actividad en el historial. El sistema genera notificaciones internas (servicio de hoy, urgente, completado, nuevo cliente, solicitud de atención) que pueden leerse o eliminarse.

5.8.5 Panel y configuración

El panel de inicio resume estadísticas, servicios del día y metas semanales. El perfil permite editar los datos del negocio, el modo oscuro, las alertas y la prueba de conexión con Make.

5.8.6 Asistente de IA y atención

El chat responde con Gemini usando como contexto los servicios del usuario. Si un cliente expresa una queja, una cancelación, un problema o pide hablar con una persona, el sistema crea una solicitud de atención con prioridad y radicado, que el administrador toma, responde y resuelve.

5.8.7 Integración con Make

Si se configura una URL, la aplicación envía un evento al crear, actualizar o completar un servicio, de modo que el negocio pueda automatizar avisos y reportes. Sin URL, la función queda desactivada.

5.9 Flujo principal de uso

1. El cliente se registra o inicia sesión y llega al Portal del cliente.
2. El cliente solicita un servicio o conversa con el asistente para consultar el estado de sus órdenes.
3. Si el mensaje del cliente expresa una queja, cancelación o problema, el sistema crea una solicitud de atención y le informa el radicado.
4. El administrador inicia sesión, revisa el panel y la atención requerida, y toma, responde y resuelve las solicitudes.
5. El administrador gestiona clientes y órdenes, cambia los estados y consulta el historial y las estadísticas.
6. Al crear o completar una orden, el sistema notifica a Make (si está configurado) para automatizar avisos.

5.10 Diferenciadores

- Un solo producto para gestionar clientes, órdenes y atención, pensado para negocios pequeños.
- Escalamiento explícito de la IA a una persona, con prioridad y radicado.
- Sin clave de IA en el APK: el acceso a Gemini se resuelve en el proyecto Firebase.
- Reglas de seguridad de mínimo privilegio y rol inmutable.
- Arquitectura por capas con pruebas ejecutables en la JVM y en integración continua.
- Integración opcional con Make sin infraestructura propia.

5.11 Capturas de la aplicación

Las siguientes figuras muestran la interfaz. Son **renders de Robolectric con datos de demostración** generados a partir del código de la aplicación con repositorios simulados; no son capturas de un dispositivo físico ni muestran datos reales.

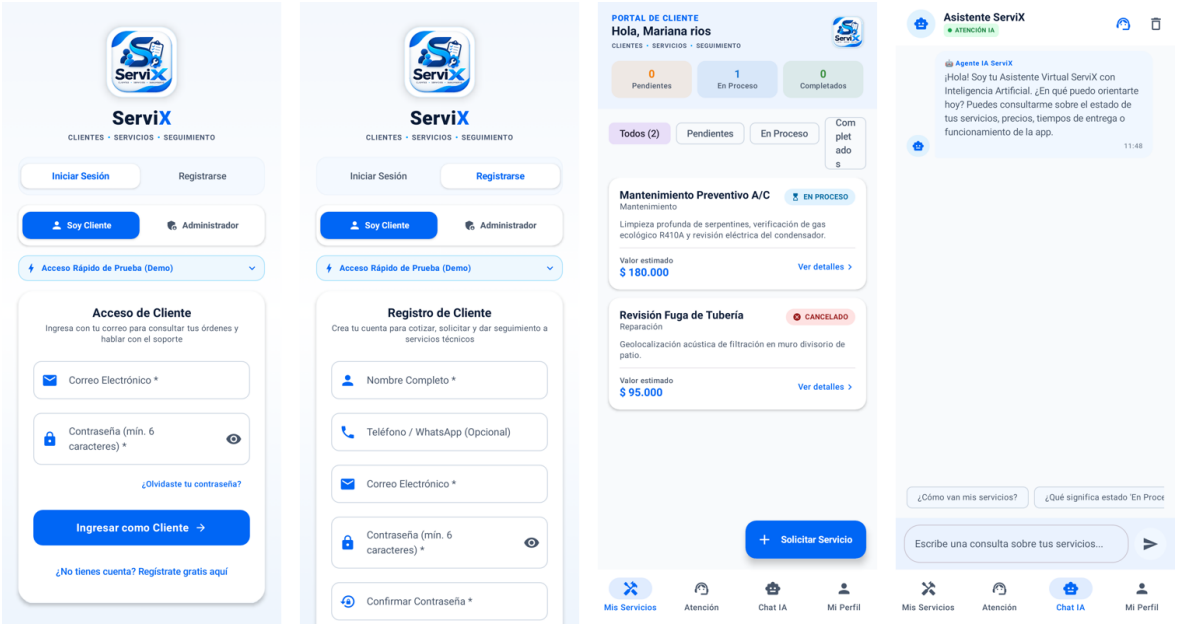


Figura 3. Acceso, registro, Portal del cliente y chat del cliente

Fuente: Render de Robolectric con datos de demostración (09_Evidencias/Capturas_App).

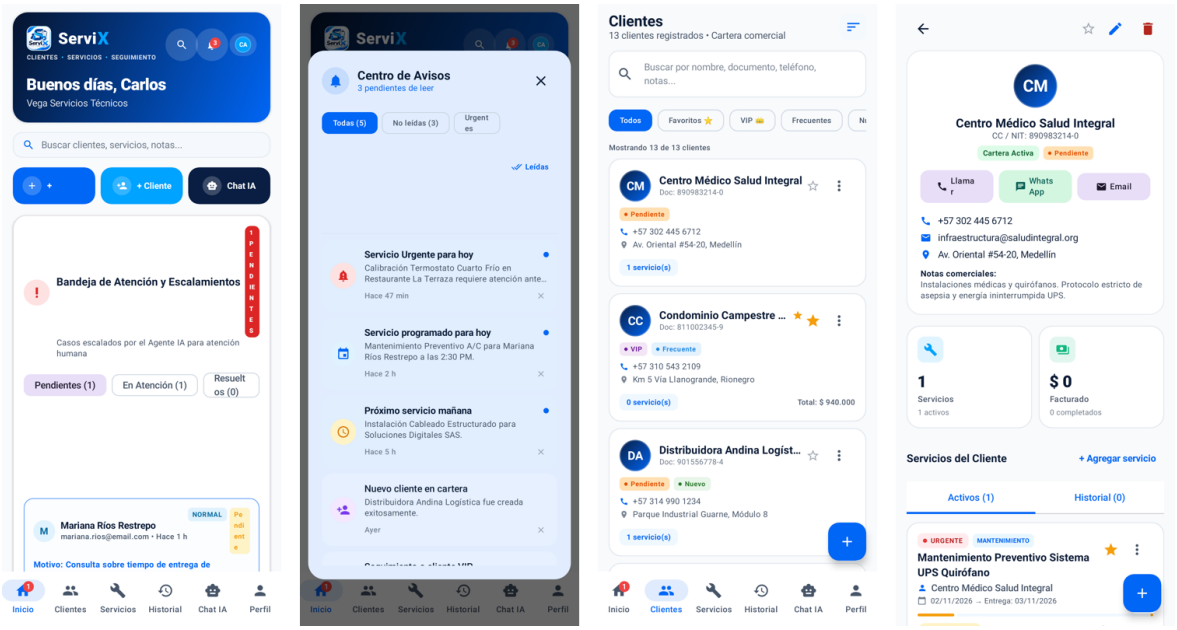


Figura 4. Inicio, notificaciones, lista de clientes y detalle de cliente

Fuente: Render de Robolectric con datos de demostración (09_Evidencias/Capturas_App).

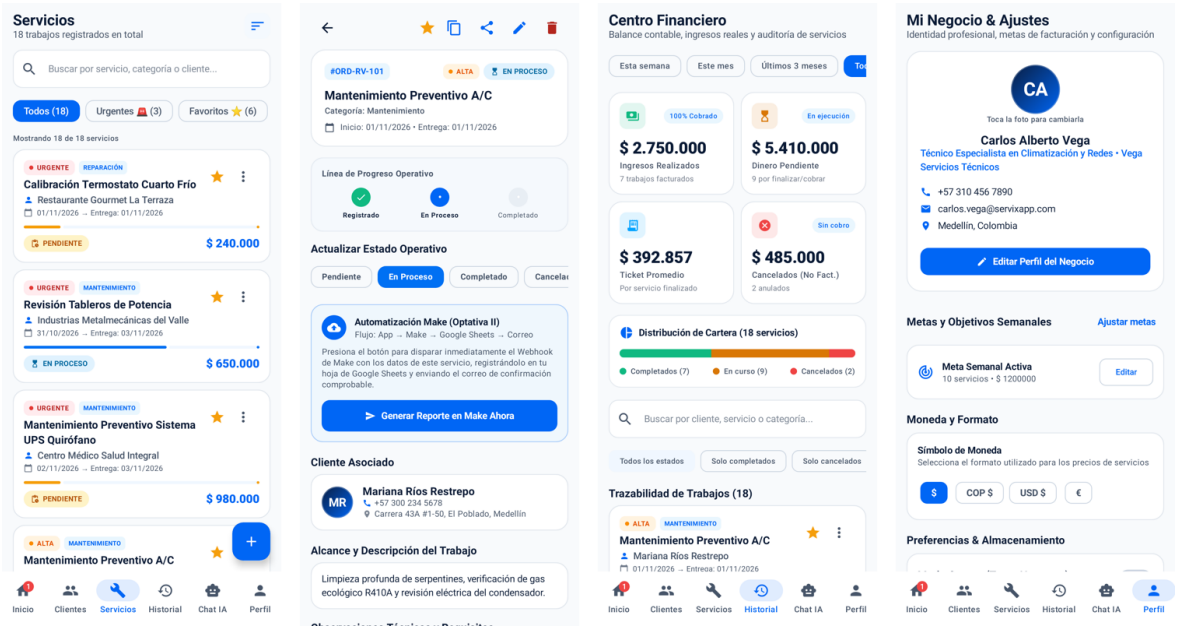


Figura 5. Lista de servicios, detalle de servicio, historial y perfil

Fuente: Render de Robolectric con datos de demostración (09_Evidencias/Capturas_App).

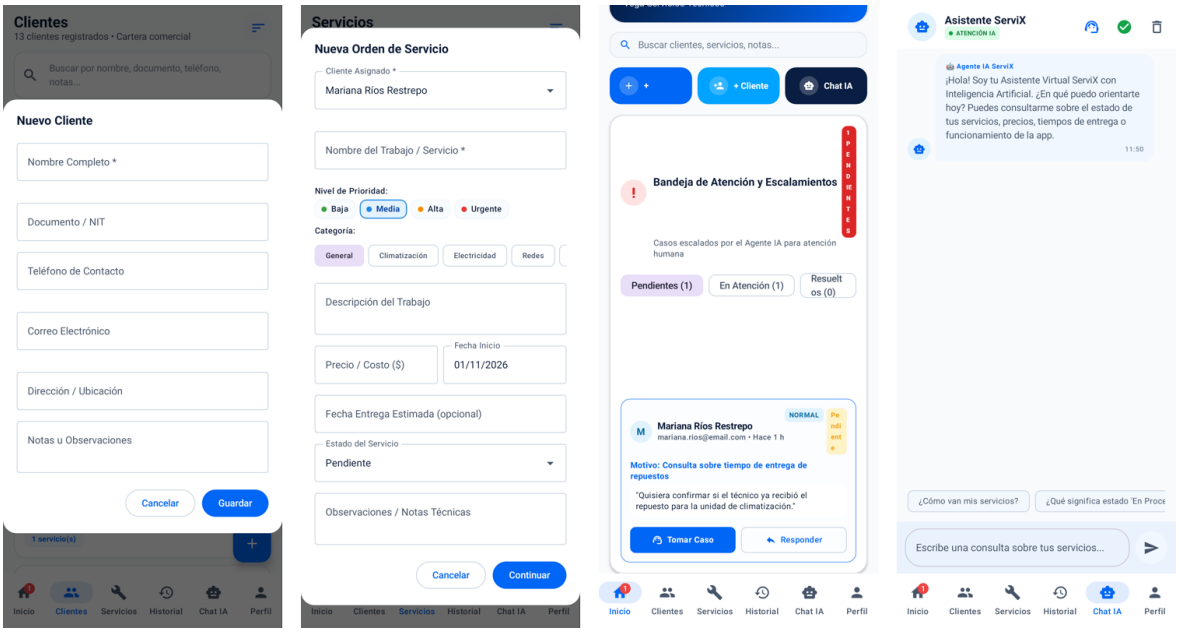


Figura 6. Formularios de alta, gestión de solicitudes de atención y chat del administrador

Fuente: Render de Robolectric con datos de demostración (09_Evidencias/Capturas_App).

6 Marco teórico

Este marco resume los fundamentos conceptuales que sustentan las decisiones del proyecto. Las citas siguen el formato APA y sus referencias completas figuran al final del documento.

6.1 Sistemas de información y gestión de servicios

Un sistema de información reúne personas, procesos, datos y tecnología para capturar, almacenar, procesar y distribuir información que apoya la toma de decisiones (Sommerville, 2016). En un negocio de servicios técnicos, los datos relevantes son los clientes, las órdenes de trabajo, su estado y su historial. Un sistema de gestión de servicios ordena ese ciclo: solicitud, programación, ejecución, cierre y atención posterior.

La ingeniería de requisitos es la disciplina que descubre, analiza, especifica y valida lo que el sistema debe hacer. La norma ISO/IEC/IEEE 29148 (2018) exige que cada requisito sea necesario, inequívoco, verificable y trazable, criterios que este documento aplica mediante fichas, criterios de aceptación y matrices.

6.2 Desarrollo móvil nativo en Android

El desarrollo nativo emplea el lenguaje y las herramientas oficiales de la plataforma. En Android, Google recomienda Kotlin como lenguaje principal y Jetpack Compose como kit de interfaz declarativa, en el que la pantalla es una función del estado (Google, s. f.-a). Material Design 3 aporta componentes y un sistema de temas coherente. La elección nativa favorece el rendimiento, el acceso a la plataforma y la madurez de las herramientas de prueba, a cambio de mantener un cliente por plataforma.

6.3 Arquitectura por capas, MVVM y SOLID

La arquitectura por capas separa responsabilidades: presentación, dominio y datos. Martin (2017) propone que las dependencias apunten hacia las reglas de negocio, de modo que los detalles (bases de datos, interfaces) puedan cambiar sin afectar al núcleo. MVVM separa la vista del estado y la lógica de presentación que vive en el ViewModel, y la guía de arquitectura de Android lo recomienda junto con repositorios y flujos de datos (Google, s. f.-a).

Los principios SOLID orientan el diseño: responsabilidad única, abierto/cerrado, sustitución de Liskov, segregación de interfaces e inversión de dependencias (Martin, 2017). ServiX aplica la inversión de dependencias con interfaces de repositorio en el dominio e implementaciones intercambiables (Firebase y simuladas).

6.4 Bases de datos NoSQL orientadas a documentos

Las bases de datos de documentos almacenan registros jerárquicos con esquema flexible y se escalan sin las uniones propias del modelo relacional. Cloud Firestore organiza los datos en colecciones y documentos, ofrece escuchas en tiempo real y reglas de seguridad declarativas que se evalúan en el servidor (Google, s. f.-b). La contrapartida es que las consultas son más limitadas que en SQL y que el modelo debe diseñarse según los patrones de acceso. ServiX usa un modelo multiusuario por identificador, que aísla los datos de cada cuenta.

6.5 Inteligencia artificial generativa y modelos de lenguaje

Los modelos de lenguaje de gran tamaño se basan en la arquitectura Transformer (Vaswani et al., 2017) y generan texto a partir de una instrucción y un contexto; con ejemplos o instrucciones adecuadas pueden resolver tareas sin entrenamiento específico (Brown et al., 2020). Sus límites son conocidos: pueden producir información incorrecta con apariencia de certeza, dependen del contexto que se les entrega y su uso con datos personales exige cautela.

Por ello ServiX combina el modelo con reglas deterministas: la clasificación de intención por palabras clave decide el escalamiento a una persona sin depender del modelo, y la instrucción de sistema prohíbe inventar datos y exponer información de otros clientes. El patrón de **humano en el circuito** reconoce que ciertos casos (quejas, cancelaciones) requieren criterio humano.

6.6 Scrum

Scrum es un marco ágil para gestionar trabajo complejo mediante iteraciones cortas (Sprints), tres roles (Product Owner, Scrum Master y Developers), artefactos (Product Backlog, Sprint Backlog, Incremento) y eventos (planificación, revisión y retrospectiva) (Schwaber y Sutherland, 2020). El proyecto adopta Scrum con Sprints de dos semanas, como pide la guía de la asignatura, y lo gestiona en Azure Boards.

6.7 DevOps e integración continua

DevOps integra el desarrollo y la operación mediante automatización y retroalimentación rápida (Kim et al., 2016). La integración continua consiste en integrar los cambios con frecuencia y verificarlos automáticamente con una compilación y un conjunto de pruebas (Humble y Farley, 2010). En ServiX, cada cambio a `main` y cada Pull Request ejecutan el pipeline ServiX-CI, que compila, prueba y publica el APK de depuración. La entrega continua a producción no está implementada.

6.8 Pruebas de software

Las pruebas buscan evidencia de que el software cumple sus requisitos y de que no hay regresiones. Se organizan por niveles (unitarias, de integración, de sistema y de aceptación) y por tipos (funcionales, de seguridad, de usabilidad). La familia ISO/IEC/IEEE 29119 define procesos y documentación de pruebas (ISO/IEC/IEEE, 2013/2021). Una pirámide de pruebas con muchas pruebas rápidas de dominio y pocas de interfaz es la estrategia que sigue ServiX (véase DOC-03).

7 Marco técnico y tecnológico

7.1 Stack tecnológico

Las versiones se leyeron de `gradle/libs.versions.toml` y de `app/build.gradle.kts` del repositorio.

Tabla 15. Stack tecnológico y versiones

Capa o función	Tecnología	Versión	Rol en ServiX
Lenguaje	Kotlin	2.2.10	Código de la aplicación.
Compilación	Android Gradle Plugin / Gradle	9.1.1 / 9.3.1	Construcción del APK.
Interfaz	Jetpack Compose (BOM) y Material 3	2024.09.00	Pantallas declarativas.
Navegación	Navigation Compose	2.8.9	Rutas entre pantallas.
Estado	Lifecycle y ViewModel Compose	2.8.7	Estado de la interfaz.
Asincronía	Kotlinx Coroutines	1.10.2	Operaciones concurrentes y flujos.
Backend gestionado	Firebase BoM (Auth, Firestore, AI Logic, App Check)	34.17.0	Identidad, datos y asistente.
Red	OkHttp y Retrofit	4.10.0 / 2.12.0	Envío de eventos a Make.
Pruebas	JUnit, Robolectric, Roborazzi	4.13.2 / 4.16.1 / 1.59.0	Pruebas en la JVM.
Plataforma	minSdk / targetSdk / compileSdk	26 / 36 / 36	Versiones de Android soportadas.
Java	JDK	21 (CI y pruebas)	Requerido por Robolectric con SDK 36.
CI/CD	Azure Pipelines, agente `ubuntu-latest`	—	Compilación, pruebas y artefacto.
Gestión	Azure DevOps (Boards, Repos, Pipelines)	—	Scrum y control de versiones.
Automatización	Make (opcional)	—	Recepción de eventos por webhook.

7.2 Alternativas consideradas

Tabla 16. Alternativas consideradas y razón de elección

Decisión	Alternativas	Elección y razón
Plataforma móvil	.NET MAUI, Flutter, aplicación web	Android nativo con Kotlin: herramientas oficiales, mejor soporte de pruebas en la JVM y experiencia previa del Developer.
Interfaz	Vistas XML	Jetpack Compose: interfaz declarativa y recomendada por Google.
Base de datos	MySQL o SQL Server con API propia, Realtime Database	Firestore: sin servidor propio, tiempo real y reglas declarativas; consultas limitadas como contrapartida.
Autenticación	Sistema propio	Firebase Authentication: gestión segura de credenciales sin desarrollar infraestructura.
Acceso a IA	API de Gemini con clave en la app, backend propio	Firebase AI Logic: evita la clave en el APK (hallazgo H-02).
Inyección de dependencias	Hilt, Koin	Manual con `AppContainer`: menos dependencias y compilación más ligera en un equipo de 4 GB.

Decisión	Alternativas	Elección y razón
Automatización	n8n, Retell	Make: configuración sencilla por webhook; n8n y Retell quedan como contratos futuros.
CI/CD	GitHub Actions	Azure Pipelines: lo exige la guía de la asignatura.
Pruebas de interfaz	Emulador con pruebas instrumentadas	Robolectric: el equipo no admite virtualización.

7.3 Arquitectura

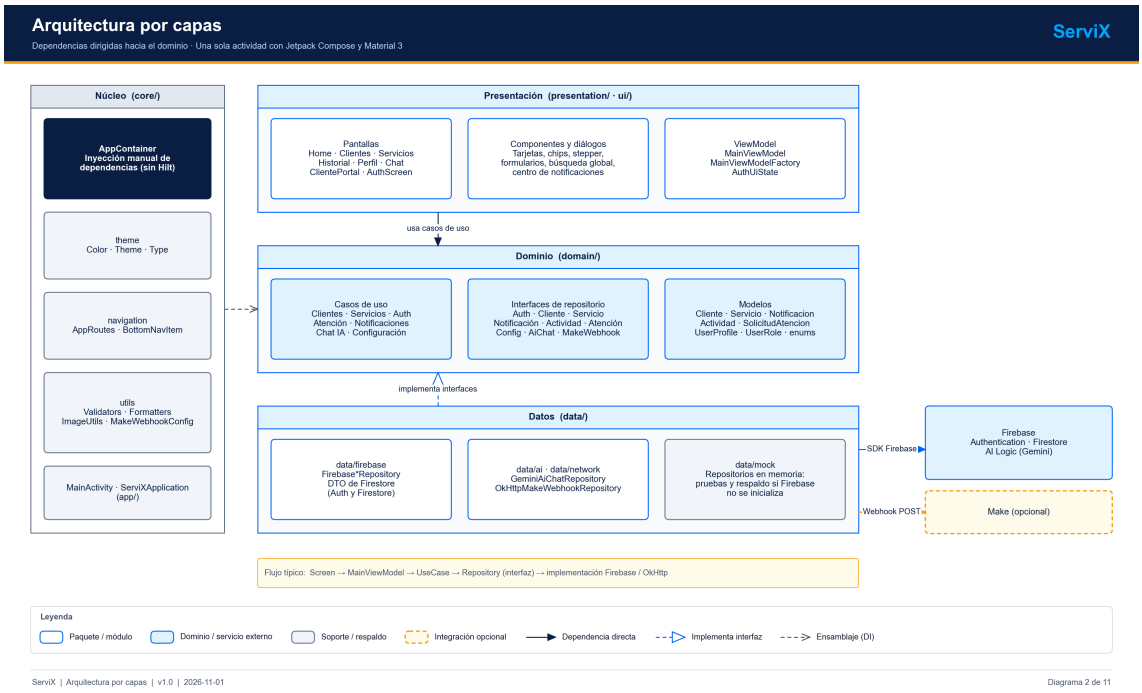


Figura 7. Arquitectura por capas de ServiX

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

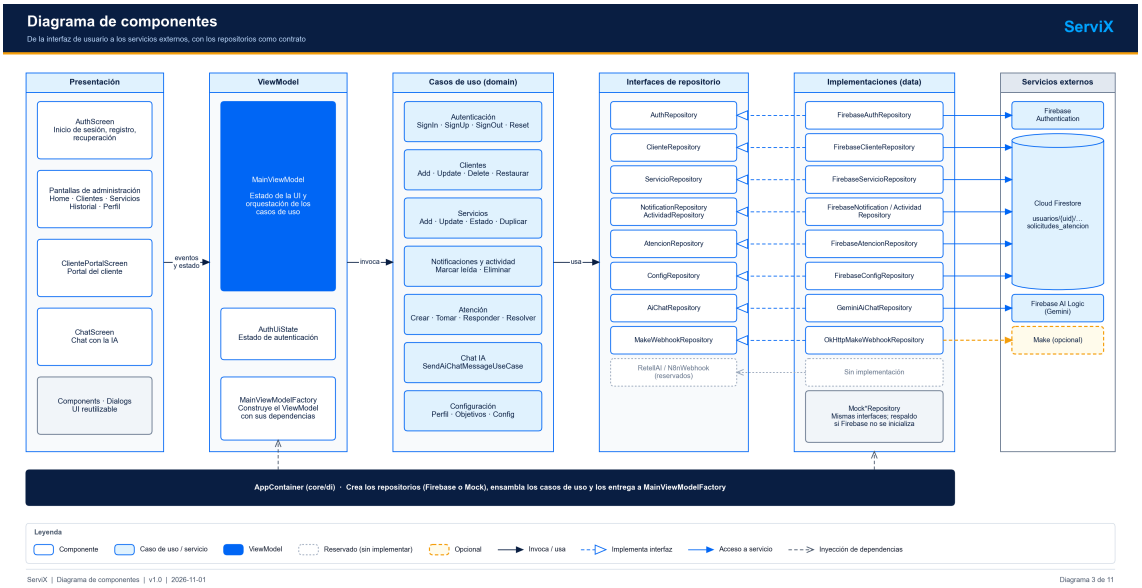


Figura 8. Componentes principales y sus dependencias

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

La descripción completa de la arquitectura, con vistas, decisiones (ADR) y diseño detallado, se encuentra en el documento DOC-02.

7.4 Seguridad técnica

- Autenticación con Firebase Authentication y rol almacenado en Firestore.
- Reglas de Firestore de mínimo privilegio con denegación por defecto y límites de longitud en las solicitudes.
- Sin claves de IA en el APK ni secretos en el repositorio; la URL de Make se inyecta al compilar.
- Atajos de acceso solo en compilaciones de depuración.
- Pendientes: App Check, restricción de la clave de API, asignación del rol de administrador desde un entorno de confianza y pruebas de las reglas.

7.5 CI/CD

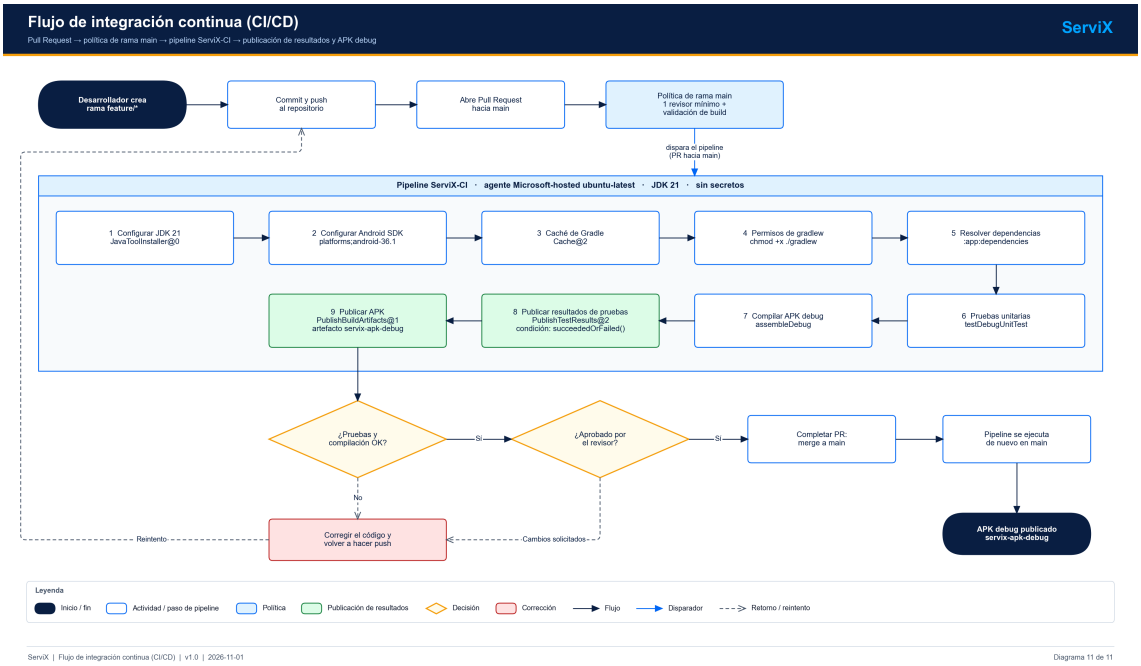


Figura 9. Flujo de integración continua en Azure Pipelines

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

El pipeline `ServiX-CI` instala JDK 21, configura el SDK de Android, restaura la caché de Gradle, ejecuta las pruebas unitarias, compila el APK de depuración y publica los resultados y el artefacto `servix-apk-debug`.

8 Marco conceptual

El glosario reúne los conceptos necesarios para leer este documento.

Tabla 17. Glosario de conceptos

Término	Definición
Aplicación nativa	Aplicación desarrollada con el lenguaje y las herramientas oficiales de una plataforma.
APK	Paquete instalable de una aplicación Android.
AppContainer	Objeto de ServiX que compone repositorios y casos de uso (inyección manual).
Autenticación	Proceso que verifica la identidad de un usuario.
Autorización	Proceso que determina qué puede hacer un usuario autenticado.
Backlog	Lista priorizada de trabajo pendiente de un producto.
BuildConfig	Clase generada en la compilación con constantes, por ejemplo la URL de Make.
Caso de uso	Descripción de una interacción entre un actor y el sistema; en el código, una clase del dominio que encapsula una operación.
CI/CD	Integración y entrega continuas.
Cloud Firestore	Base de datos de documentos en la nube con escuchas en tiempo real.

Término	Definición
Compose	Kit de Android para construir interfaces de forma declarativa.
Corrutina	Mecanismo de Kotlin para escribir código asíncrono de forma secuencial.
DTO	Objeto de transferencia entre el dominio y los documentos de Firestore.
Escalamiento	Traslado de un caso del asistente de IA a una persona.
Firebase AI Logic	Servicio de Firebase que permite usar modelos Gemini desde una aplicación sin incluir la clave.
Gemini	Familia de modelos de lenguaje de Google.
Habeas data	Derecho de las personas a conocer, actualizar y rectificar la información que sobre ellas se recopile.
Historia de usuario	Descripción breve de una necesidad desde el punto de vista del usuario.
Humano en el circuito	Diseño en el que una persona interviene en las decisiones de un sistema automático.
Intención	Propósito que el sistema infiere de un mensaje del usuario.
LLM	Modelo de lenguaje de gran tamaño.
Make	Plataforma de automatización de flujos mediante webhooks.
Material 3	Sistema de diseño de Google.
Mínimo privilegio	Principio por el cual cada usuario recibe solo los permisos necesarios.
Mock	Implementación simulada en memoria de una dependencia.
MVVM	Patrón que separa Modelo, Vista y ViewModel.
NoSQL	Bases de datos no relacionales, como las orientadas a documentos.
PBI	Product Backlog Item de Azure Boards.
Pipeline	Secuencia automática de pasos de compilación y verificación.
Pull Request	Solicitud de integrar una rama, sujeta a revisión.
Radicado	Código de 8 caracteres de una solicitud de atención.
Repositorio (patrón)	Interfaz que abstrae el acceso a los datos.
Robolectric	Marco para ejecutar pruebas de Android en la JVM.
Scrum	Marco ágil de gestión basado en Sprints.
Sprint	Iteración de duración fija en Scrum (dos semanas en el proyecto).
SRS	Especificación de requisitos de software.
Transmisión y transferencia de datos	Envío de datos personales a un encargado o responsable ubicado fuera del país.
Webhook	URL que recibe una petición HTTP cuando ocurre un evento.

9 Marco legal y normativo

9.1 Alcance y advertencia

> **Advertencia. Análisis académico, no asesoría jurídica**
> Este capítulo es un análisis académico del equipo. No constituye asesoría jurídica. Solo se citan normas que el equipo conoce con certeza y su **vigencia, alcance y modificaciones deben verificarse en fuentes oficiales** (por ejemplo, la Secretaría Jurídica Distrital, la Superintendencia de Industria y Comercio y la Presidencia de la República) antes de operar el sistema con datos reales.

9.2 Protección de datos personales

La **Ley 1581 de 2012** dicta disposiciones generales para la protección de datos personales y desarrolla el derecho constitucional de habeas data. Establece principios (legalidad, finalidad, libertad, veracidad, transparencia, acceso y circulación restringida, seguridad y confidencialidad), derechos de los titulares, la necesidad de obtener autorización previa, expresa e informada para el tratamiento, y deberes para el responsable y el encargado del tratamiento. El **Decreto 1377 de 2013**, compilado en el **Decreto 1074 de 2015**, reglamenta aspectos como la autorización, las políticas de tratamiento y la transmisión de datos. La **Superintendencia de Industria y Comercio (SIC)** es la autoridad de protección de datos y administra el **Registro Nacional de Bases de Datos (RNBD)**, cuya obligación de inscripción depende de las condiciones que la normativa defina para cada responsable y debe verificarse. La **Ley 1266 de 2008** regula el habeas data financiero, comercial y crediticio y no es central para ServiX.

ServiX trata datos personales de clientes (nombre, correo, teléfono, documento de identidad, dirección) y de los usuarios de la aplicación (correo, nombre, teléfono). Si el negocio que use ServiX es responsable del tratamiento, deberá contar con autorización de los titulares y con una política de tratamiento. Además, el uso de Firebase y de Gemini implica enviar datos a servicios de terceros cuyos servidores pueden estar fuera de Colombia, lo que puede constituir una transmisión o transferencia internacional de datos y debe analizarse.

9.3 Delitos informáticos

La **Ley 1273 de 2009** modificó el Código Penal para proteger la información y los datos como bien jurídico y tipifica conductas como el acceso abusivo a un sistema informático, la interceptación de datos y la violación de datos personales. Para ServiX implica la necesidad de controlar los accesos, proteger los datos y no exponer credenciales; las reglas de Firestore y la ausencia de secretos en el repositorio responden a ello.

9.4 Propiedad intelectual y licencias

La **Ley 23 de 1982** sobre derechos de autor, modificada por la **Ley 1915 de 2018**, y la **Decisión Andina 351 de 1993** (régimen común sobre derecho de autor y derechos conexos) protegen las obras, entre ellas los programas de ordenador. El código de ServiX es obra de sus autores; el repositorio **no contiene un archivo `LICENSE`**, de modo que no declara las condiciones de uso y distribución. Además, el proyecto usa dependencias de código abierto con sus propias licencias.

Tabla 18. Licencias de las dependencias utilizadas (a verificar en cada proyecto)

Dependencia	Licencia habitual	Observación
AndroidX, Jetpack Compose, Material (iconos), Navigation, Lifecycle	Apache License 2.0	Permite uso y distribución con atribución y aviso de licencia.
Kotlin y Kotlinx Coroutines	Apache License 2.0	Ídem.

Dependencia	Licencia habitual	Observación
OkHttp y Retrofit (Square)	Apache License 2.0	Ídem.
Firebase SDK para Android	Apache License 2.0 (código) y términos de servicio de Firebase	El uso del servicio se rige por los términos de Google.
JUnit 4	Eclipse Public License 1.0	Solo en pruebas.
Robolectric	MIT License	Solo en pruebas.
Roborazzi	Apache License 2.0	Solo en pruebas.

9.5 Mensajes de datos y comercio electrónico

La **Ley 527 de 1999** define y reglamenta el acceso y uso de los mensajes de datos, el comercio electrónico y las firmas digitales, y reconoce efectos jurídicos a los mensajes de datos. En ServiX, los registros de solicitudes y actividades son mensajes de datos; no se implementan firmas electrónicas. La **Ley 1480 de 2011** (Estatuto del Consumidor) sería aplicable en lo relativo a información veraz y suficiente si el negocio ofrece sus servicios a consumidores a través de la aplicación; ServiX no procesa pagos.

9.6 Términos de servicio de terceros

- **Google y Firebase:** el uso de Authentication, Firestore y AI Logic se rige por los términos de servicio de Firebase y de Google Cloud, incluidos sus anexos de tratamiento de datos.
- **API de Gemini:** el uso se rige por los términos adicionales de la API de Gemini; su tratamiento de datos (por ejemplo, si las entradas pueden usarse para mejorar los productos) depende del servicio y del plan, y debe revisarse antes de enviar datos de clientes.
- **Azure DevOps y GitHub:** el uso se rige por los términos de Microsoft y de GitHub.
- **Make:** el tratamiento de los datos de los eventos depende de la cuenta y de los términos de Make.

9.7 Uso responsable de la inteligencia artificial

ServiX aplica prácticas de uso responsable: informa al usuario que conversa con un asistente de IA, no atribuye al modelo decisiones definitivas, permite el escalamiento a una persona, prohíbe en la instrucción de sistema inventar datos o exponer información de otros clientes y limita el contexto enviado. Pendientes: avisar explícitamente que los mensajes se envían a un proveedor de IA, definir un periodo de retención y evaluar sesgos y errores.

9.8 Estándares técnicos

Tabla 19. Estándares técnicos de referencia

Estándar	Aplicación en ServiX
ISO/IEC 25010	Clasificación de los requisitos no funcionales por característica de calidad.
ISO/IEC 27001 y 27002	Referencia para la gestión de la seguridad de la información y sus controles; ServiX no está certificado.
OWASP MASVS	Referencia de seguridad para aplicaciones móviles (almacenamiento, red, autenticación, código).

Estándar	Aplicación en ServiX
IEEE 29148, 830, 1016, 29119	Estructura del SRS, correspondencia, diseño y documentación de pruebas.
ISO/IEC/IEEE 42010	Descripción de la arquitectura.

9.9 Obligaciones, tratamiento actual y brechas

Tabla 20. Obligación, cómo la aborda ServiX hoy y brecha o acción pendiente

Obligación o referencia	Cómo la aborda ServiX hoy	Brecha o acción pendiente
Autorización previa del titular (Ley 1581, Decreto 1377/1074)	No implementada: la aplicación no solicita ni registra autorización.	Diseñar y registrar la autorización al crear clientes y cuentas.
Política de tratamiento de datos	No existe.	Redactar y publicar la política; informar finalidades, derechos y canales.
Finalidad y minimización	Se recogen nombre, documento, teléfono, correo, dirección y notas; el documento no siempre es necesario.	Revisar qué campos son necesarios y hacerlos opcionales.
Seguridad de la información	Reglas de mínimo privilegio; sin secretos en el repositorio; HTTPS de Firebase.	Probar reglas, activar App Check, asignar roles desde servidor.
Derechos del titular (acceso, actualización, supresión)	El administrador puede editar y eliminar clientes; no hay canal para que el titular lo solicite.	Definir el procedimiento de atención de consultas y reclamos.
Transmisión o transferencia internacional	Los datos se almacenan y procesan en servicios de Google; la ubicación no se documenta.	Documentar la ubicación y los contratos o cláusulas aplicables.
Registro Nacional de Bases de Datos	No evaluado.	Verificar si el responsable está obligado a inscribirse ante la SIC.
Delitos informáticos (Ley 1273)	Controles de acceso y ausencia de credenciales expuestas.	Registrar incidentes de seguridad y definir un procedimiento de respuesta.
Derechos de autor y licencias (Ley 23/1982, Ley 1915/2018, Decisión 351)	El código es de los autores; se usan dependencias con licencias permisivas.	Añadir un archivo LICENSE y un aviso de licencias de terceros.
Mensajes de datos (Ley 527)	Los registros son electrónicos; sin firma electrónica.	Evaluar si se requiere firma o sello de tiempo.
Estatuto del consumidor (Ley 1480), si aplica	No se procesan pagos ni publicidad.	Revisar si el negocio que use la app tiene deberes de información.
Términos de Google/Gemini	El uso es acorde con el plan del proyecto; el tratamiento de datos de la IA no se ha revisado.	Revisar términos y evitar enviar datos innecesarios al modelo.
Uso responsable de IA	Escalamiento a humano, reglas de no invención y contexto acotado.	Avisar al usuario del envío de datos; evaluar errores y sesgos.

10 Consideraciones éticas

El equipo adopta como referencia los códigos de ética profesional de la ACM (2018) y del IEEE, que orientan a los ingenieros de software a actuar en interés público, evitar el daño, ser honestos y respetar la privacidad.

Tabla 21. Consideraciones éticas y su aplicación

Tema	Compromiso	Aplicación en el proyecto
Honestidad técnica	No afirmar lo que no se verificó.	Los documentos distinguen lo ejecutado de lo no ejecutado y declaran las limitaciones.
Privacidad	Tratar los datos personales con cuidado.	Datos ficticios en pruebas y capturas; reconocimiento de la brecha de autorización y política de tratamiento.
Seguridad	Proteger a los usuarios.	Reglas de mínimo privilegio, sin secretos en el repositorio; riesgos abiertos documentados.
Transparencia de la IA	No hacer pasar respuestas automáticas por humanas.	El asistente se identifica como Agente Virtual; en fallos no se simulan respuestas de IA.
Equidad y accesibilidad	Evitar excluir usuarios.	Interfaz en español con modo oscuro; sin pruebas de accesibilidad aún.
Integridad académica	Atribuir el trabajo ajeno.	Las dependencias y fuentes se citan; el código y la documentación son del equipo.
Uso de asistentes de IA en el desarrollo	Revisar y responsabilizarse del resultado.	El código y los documentos fueron revisados por el equipo, que responde por ellos.

11 Metodología

11.1 Marco de trabajo

El proyecto sigue Scrum, gestionado en Azure DevOps (organización Serivx, proyecto ServiX). Cada Sprint dura dos semanas. La metodología combina un enfoque iterativo para construir y verificar el producto con prácticas de integración continua.

Tabla 22. Roles Scrum del equipo

Integrante	Rol	Responsabilidad
Nicolas Quiroz Gomez	Product Owner	Prioriza el backlog y define el valor.
Juan Sebastian Garzon Suarez	Scrum Master	Facilita el proceso y vela por el pipeline.
Juan Esteban Olaya Muñoz	Developer	Implementa, prueba y documenta.

11.2 Sprints

Tabla 23. Sprints (fechas provisionales)

Sprint	Fechas	Objetivo	PBI	Tareas
Sprint 1	02/11/2026 - 13/11/2026	Análisis, requisitos y diseño: SRS, arquitectura, modelo de datos, diagramas y plan de pruebas; revisión de la auditoría.	3 (según captura del backlog)	6

Sprint	Fechas	Objetivo	PBI	Tareas
Sprint 2	16/11/2026 - 27/11/2026	Desarrollo de funcionalidades principales: Autenticación con roles, clientes, servicios, historial, notificaciones y chat con IA.	5 (según captura del backlog)	14
Sprint 3	30/11/2026 - 11/12/2026	Integración, pruebas y despliegue: Pipeline CI/CD, pruebas unitarias e informe, documentación y presentación final.	4 (por diferencia con el total de 12)	7 (por diferencia con el total de 27)

11.3 Backlog

El Product Backlog contiene 12 PBI, 27 tareas y 6 bugs. Los bugs proceden de la auditoría técnica del código (Bug 42 a Bug 47); cuatro están resueltos y dos siguen abiertos (Bug 45 y Bug 46).

Tabla 24. Elementos principales del backlog

ID	Título	Esfuerzo	Estado
PBI 10	Autenticación con roles Cliente y Administrador	8	Committed
PBI 14	Gestión de clientes (alta, edición, borrado, restauración)	8	Committed
PBI 18	Gestión de servicios y seguimiento de estados		Committed
—	Historial y notificaciones		Committed
—	Chat con IA y escalamiento a administrador		Committed
PBI 32	Pipeline CI/CD en Azure Pipelines	5	Approved
PBI 35	Pruebas unitarias e informe de pruebas	8	Approved
—	Documentación técnica, evidencias y presentación final		Committed

11.4 Definición de terminado

- El código está integrado en `main` mediante un Pull Request revisado por al menos una persona.
- El pipeline ServiX-CI pasa: pruebas aprobadas y APK compilado.
- No se incluyen secretos y la documentación afectada está actualizada.
- Los criterios de aceptación del elemento se han comprobado o se declaran como no ejecutados.
- El elemento está en Done en el tablero.

11.5 Herramientas y flujo de ramas

Tabla 25. Herramientas del proyecto

Herramienta	Uso
Azure Boards	Backlog, Sprints, tablero y dashboard.
Azure Repos y GitHub	Control de versiones; el repositorio público del código es https://github.com/JuanXD2201/ServiX.

Herramienta	Uso
Azure Pipelines	Integración continua `ServiX-CI`.
Android Studio, Visual Studio Code, Git	Desarrollo.
Firebase y Make	Servicios de la aplicación.

Se usan las ramas `main`, `develop` y `feature/<descripcion>`. Cada cambio se integra mediante Pull Request hacia `main` con política de un revisor; los PR #1 y #2 corrigieron el pipeline. Los mensajes de commit siguen prefijos de tipo (`feat`, `fix`, `docs`, `chore`, `refactor`, `security`, `ci`).

11.6 Gestión de riesgos

Los riesgos se registran con probabilidad e impacto (Anexo B) y se revisan en cada Sprint. Los más relevantes son la autoasignación del rol de administrador, la degradación silenciosa a repositorios simulados, la falta de pruebas en dispositivo y las brechas de protección de datos personales.

12 Requisitos específicos

12.1 Requisitos funcionales

Los requisitos funcionales se agrupan por módulo. Cada uno se presenta en una ficha con su descripción, entradas, proceso, salidas, prioridad, criterio de aceptación verificable, origen en el código y estado de verificación. El estado coincide con el de la matriz maestra del proyecto.

12.1.1 Autenticación y roles

Tabla 26. Fichas de requisitos funcionales: autenticación y roles (RF-001 a RF-005)

Campo	Contenido
RF-001	**Registro de cuenta** · Prioridad **Alta** · Estado: **Parcialmente cumplido**
Descripción	El usuario crea una cuenta con nombre, correo, contraseña, teléfono opcional y rol.
Entradas	Nombre completo, correo, contraseña (6 o más caracteres), teléfono opcional y rol elegido.
Proceso	Valida los campos en `SignUpUseCase`; crea la cuenta en Firebase Authentication; actualiza el nombre visible; sincroniza el perfil en `usuarios/{uid}` con el rol y la fecha de creación.
Salidas	`AuthResult.Success` con el usuario creado, o `AuthResult.Error` con un mensaje comprensible.
Criterio de aceptación	Se rechaza nombre vacío, correo sin '@' y contraseña de menos de 6 caracteres; con datos válidos se crea la cuenta y el perfil usuarios/{uid} con su rol.
Origen en el código	`ui/screens/AuthScreen.kt`, `SignUpUseCase`, `FirebaseAuthRepository`
Historia y pruebas	HU-001; CP-016, CP-033
RF-002	**Inicio de sesión con validación de rol** · Prioridad **Alta** · Estado: **Parcialmente cumplido**
Descripción	El usuario inicia sesión con correo y contraseña; el rol se toma de la cuenta almacenada y, si no coincide con el elegido, se rechaza el acceso.
Entradas	Correo, contraseña y rol seleccionado en pantalla.
Proceso	Valida que el correo no esté vacío y que la contraseña tenga 6 o más caracteres; autentica en Firebase; lee el rol almacenado; si el rol seleccionado no coincide cierra la sesión y rechaza el acceso.
Salidas	Usuario autenticado con su rol almacenado, o mensaje de error (credenciales, longitud o perfil equivocado).
Criterio de aceptación	Contraseña de menos de 6 caracteres devuelve 'La contraseña debe tener al menos 6 caracteres'; un rol distinto al almacenado rechaza el acceso.
Origen en el código	`SignInUseCase`, `FirebaseAuthRepository (H-17, H-18)`
Historia y pruebas	HU-002; CP-014, CP-015, CP-030, CP-031, CP-032
RF-003	**Cierre de sesión** · Prioridad **Alta** · Estado: **Cumplido**
Descripción	El usuario cierra sesión desde Perfil o Portal y vuelve a la pantalla de autenticación.
Entradas	Acción de cierre de sesión desde Perfil o Portal.
Proceso	Invoca `SignOutUseCase`, que cierra la sesión en Firebase y limpia el usuario actual.
Salidas	Estado sin usuario y navegación a la ruta `auth`.

Campo	Contenido
Criterio de aceptación	Tras cerrar sesión no existe usuario actual y la navegación vuelve a la ruta auth.
Origen en el código	`SignOutUseCase`, `MainViewModel.signOut`, `PerfilScreen`, `ClientePortalScreen`
Historia y pruebas	HU-002; CP-017
RF-004	**Recuperación de contraseña** · Prioridad **Media** · Estado: **No verificable**
Descripción	El usuario solicita el envío de un correo de restablecimiento.
Entradas	Correo del usuario.
Proceso	Rechaza correo vacío; solicita a Firebase Authentication el envío del correo de restablecimiento.
Salidas	`Result.success` o `Result.failure` con el motivo; mensaje en pantalla.
Criterio de aceptación	Con correo vacío se muestra 'Ingresa tu correo para recuperar contraseña'; con correo válido se solicita el envío a Firebase.
Origen en el código	`SendPasswordResetUseCase`, `FirebaseAuthRepository.sendPasswordReset`
Historia y pruebas	HU-003; CP-034
RF-005	**Navegación según rol** · Prioridad **Alta** · Estado: **Parcialmente cumplido**
Descripción	Tras autenticarse, el cliente ve el Portal del cliente y el administrador ve el panel con barra inferior (Inicio, Clientes, Servicios, Historial, Chat IA, Perfil).
Entradas	Usuario autenticado con su rol.
Proceso	`ServiXApp` evalúa el rol: para `CLIENTE` muestra el Portal sin barra inferior; para `ADMINISTRADOR` muestra la barra con seis destinos; sin sesión muestra la autenticación.
Salidas	Árbol de navegación acorde al rol.
Criterio de aceptación	Rol CLIENTE navega a cliente_portal; rol ADMINISTRADOR navega a home; sin sesión se muestra auth.
Origen en el código	`app/MainActivity.kt`, `core/navigation/AppRoutes.kt`
Historia y pruebas	HU-002; CP-003, CP-004, CP-025, CP-026, CP-027, CP-028, CP-029, CP-030, CP-031

> **Nota. Rol y navegación**
> El rol decide la navegación pero no sustituye a las reglas de Firestore: la interfaz oculta funciones, y las reglas son el mecanismo que las hace cumplir en el servidor.

12.1.2 Gestión de clientes

Tabla 27. Fichas de requisitos funcionales: gestión de clientes (RF-006 a RF-009)

Campo	Contenido
RF-006	**Gestión de clientes: alta y edición** · Prioridad **Alta** · Estado: **Cumplido**
Descripción	El administrador registra y edita clientes (nombre, documento, teléfono, correo, dirección, notas, foto).
Entradas	Nombre, documento, teléfono, correo, dirección, notas y foto opcional.
Proceso	`AddClienteUseCase` y `UpdateClienteUseCase` validan y persisten; se registra la actividad y se genera una notificación de nuevo cliente; las iniciales se derivan del nombre.
Salidas	Cliente persistido y visible en la lista y en el detalle.

Campo	Contenido
Criterio de aceptación	El cliente creado recibe identificador, aparece en la lista y conserva sus datos; las iniciales se calculan del nombre.
Origen en el código	`AddClienteUseCase`, `UpdateClienteUseCase`, `ClientFormDialog`, `Validators`
Historia y pruebas	HU-004; CP-005, CP-006, CP-007, CP-011, CP-012, CP-035
RF-007	**Eliminación y restauración de clientes** · Prioridad **Media** · Estado: **Cumplido**
Descripción	El administrador elimina un cliente (junto con sus servicios) y puede deshacer la operación.
Entradas	Identificador del cliente a eliminar o restaurar.
Proceso	Elimina el cliente y sus servicios (borrado en cascada) y devuelve ambos para permitir **Deshacer**; la restauración vuelve a escribir cliente y servicios.
Salidas	Lista actualizada; cliente y servicios recuperables.
Criterio de aceptación	Al eliminar un cliente se eliminan sus servicios asociados; restaurar recupera cliente y servicios.
Origen en el código	`DeleteClienteUseCase`, `RestaurarClienteUseCase`, `MainViewModel.deleteCliente/restaurarCliente`
Historia y pruebas	HU-004; CP-009, CP-035
RF-008	**Favoritos** · Prioridad **Baja** · Estado: **Parcialmente cumplido**
Descripción	El administrador marca clientes y servicios como favoritos.
Entradas	Acción de marcar favorito sobre un cliente o servicio.
Proceso	Alterna el campo `esFavorito` y lo persiste.
Salidas	Indicador de estrella actualizado.
Criterio de aceptación	El indicador de favorito cambia y se persiste.
Origen en el código	`ToggleClienteFavoritoUseCase`, `ToggleServicioFavoritoUseCase`, `FavoriteStarButton`
Historia y pruebas	HU-005; CP-035, CP-036
RF-009	**Segmentación automática de clientes** · Prioridad **Media** · Estado: **Parcialmente cumplido**
Descripción	Cada cliente recibe segmentos (VIP, Frecuente, Nuevo, Pendiente, Inactivo) según sus servicios y antigüedad.
Entradas	Servicios del cliente y fecha de registro.
Proceso	`Cliente.calcularSegmentos` evalúa facturación completada, número de servicios, servicios abiertos y antigüedad.
Salidas	Lista de segmentos (VIP, Frecuente, Nuevo, Pendiente, Inactivo) mostrada como etiquetas.
Criterio de aceptación	VIP con facturación completada ≥ 500.000 o 3 servicios completados; Frecuente con 2 o más servicios; Pendiente con servicios abiertos; Nuevo con menos de 15 días y 1 servicio como máximo.
Origen en el código	`domain/model/Cliente.calcularSegmentos`, `SegmentChip`
Historia y pruebas	HU-004; CP-035

12.1.3 Gestión de servicios

Tabla 28. Fichas de requisitos funcionales: gestión de servicios (RF-010 a RF-011)

Campo	Contenido
RF-010	**Gestión de servicios** · Prioridad **Alta** · Estado: **Cumplido**
Descripción	El administrador crea, edita, duplica y elimina servicios con cliente, categoría, precio, fechas, prioridad y observaciones.
Entradas	Cliente, nombre, descripción, categoría, precio, fechas, prioridad y observaciones.
Proceso	Los casos de uso crean, actualizan, duplican, eliminan y restauran servicios; el estado inicial es Pendiente y la prioridad por defecto es Media; se registra actividad y, si procede, se notifica a Make.
Salidas	Servicio persistido y visible en la lista y en el detalle.
Criterio de aceptación	El servicio nuevo queda en estado Pendiente y con prioridad Media por defecto; duplicar crea una copia; eliminar permite restaurar.
Origen en el código	`AddServicioUseCase`, `UpdateServicioUseCase`, `DuplicarServicioUseCase`, `DeleteServicioUseCase`, `RestaurarServicioUseCase`, `ServiceFormDialog`
Historia y pruebas	HU-006; CP-005, CP-008, CP-010, CP-012, CP-036
RF-011	**Seguimiento del estado del servicio** · Prioridad **Alta** · Estado: **Cumplido**
Descripción	El administrador cambia el estado de un servicio (Pendiente, En Proceso, Completado, Cancelado) y ve el progreso en un stepper.
Entradas	Servicio y nuevo estado (Pendiente, En Proceso, Completado, Cancelado).
Proceso	`UpdateServicioEstadoUseCase` actualiza el estado, registra la actividad, genera notificación al completar y envía el evento a Make cuando el estado es Completado.
Salidas	Estado reflejado en chips y en el indicador de progreso.
Criterio de aceptación	El cambio de estado se refleja en la lista y en el detalle; al completar o cancelar se registra la actividad.
Origen en el código	`UpdateServicioEstadoUseCase`, `ServiceProgressStepper`, `StatusChip`
Historia y pruebas	HU-006; CP-008, CP-012, CP-023, CP-036

12.1.4 Historial, notificaciones y búsqueda

Tabla 29. Fichas de requisitos funcionales: historial, notificaciones y búsqueda (RF-012 a RF-014)

Campo	Contenido
RF-012	**Historial de actividades y resumen financiero** · Prioridad **Media** · Estado: **Parcialmente cumplido**
Descripción	El sistema registra actividades (alta, edición, eliminación de clientes y servicios) y las presenta en Historial junto con el Centro Financiero.
Entradas	Operaciones sobre clientes y servicios.
Proceso	Cada caso de uso relevante invoca `ActividadRepository.logActividad`; `HistorialScreen` presenta las actividades y el Centro Financiero.
Salidas	Lista cronológica de actividades.
Criterio de aceptación	Cada operación relevante genera una actividad con título, descripción y fecha.
Origen en el código	`ActividadRepository`, `HistorialScreen`, `TipoActividad`

Campo	Contenido
Historia y pruebas	HU-007; CP-028, CP-037
RF-013	**Notificaciones internas** · Prioridad **Media** · Estado: **Parcialmente cumplido**
Descripción	El sistema genera notificaciones (servicio de hoy, urgente, pendiente, completado, nuevo cliente, seguimiento, solicitud de atención) y permite marcarlas como leídas o eliminarlas.
Entradas	Eventos del sistema (servicio de hoy, urgente, pendiente, completado, nuevo cliente, seguimiento, solicitud de atención).
Proceso	Se crean documentos en `notificaciones`; el usuario puede marcar una o todas como leídas o eliminarlas.
Salidas	Centro de notificaciones con contador de no leídas.
Criterio de aceptación	Una solicitud de atención genera una notificación para el administrador; marcar como leída actualiza el contador.
Origen en el código	`NotificationRepository`, `MarcarNotificacionLeidaUseCase`, `MarcarTodasNotificacionesLeidasUseCase`, `DeleteNotificacionUseCase`, `NotificationCenterDialog`
Historia y pruebas	HU-007; CP-018, CP-037
RF-014	**Búsqueda global** · Prioridad **Media** · Estado: **No verificable**
Descripción	El administrador busca en clientes, servicios, notas y categorías desde una sola ventana.
Entradas	Texto de búsqueda.
Proceso	`GlobalSearchDialog` filtra clientes, servicios, notas y categorías en memoria.
Salidas	Lista de coincidencias o mensaje de ausencia de resultados.
Criterio de aceptación	Al escribir se listan coincidencias; sin resultados se muestra un mensaje de ausencia de coincidencias.
Origen en el código	`GlobalSearchDialog`, `HomeScreen`
Historia y pruebas	HU-008; CP-037

12.1.5 Panel de inicio, perfil y configuración

Tabla 30. Fichas de requisitos funcionales: panel de inicio, perfil y configuración (RF-015 a RF-023)

Campo	Contenido
RF-015	**Panel de inicio y metas semanales** · Prioridad **Media** · Estado: **Parcialmente cumplido**
Descripción	El administrador consulta estadísticas, servicios del día y define metas semanales de servicios e ingresos.
Entradas	Metas de servicios e ingresos semanales.
Proceso	`HomeScreen` calcula estadísticas desde los flujos de clientes y servicios; `UpdateObjetivosUseCase` guarda las metas en `configuracion/app_config`.
Salidas	Tarjetas de estadísticas y progreso de metas.
Criterio de aceptación	Las metas se guardan en la configuración (por defecto 10 servicios y 1.200.000 de ingresos).
Origen en el código	`HomeScreen`, `StatisticCard`, `WeeklyGoalsCard`, `UpdateObjetivosUseCase`, `AppConfig`
Historia y pruebas	HU-008; CP-025

Campo	Contenido
RF-016	**Perfil del negocio y configuración** · Prioridad **Media** · Estado: **Parcialmente cumplido**
Descripción	El administrador edita el perfil del negocio, foto, modo oscuro, alertas, autoguardado y moneda.
Entradas	Datos del negocio, foto (Base64), modo oscuro, alertas, autoguardado y moneda.
Proceso	`UpdateProfileUseCase` y `UpdateConfigUseCase` guardan `user_profile` y `app_config`.
Salidas	Configuración persistente y aplicada.
Criterio de aceptación	Los cambios de perfil y configuración se guardan y persisten.
Origen en el código	`PerfilScreen`, `UpdateProfileUseCase`, `UpdateConfigUseCase`, `ImageUtils`
Historia y pruebas	HU-009; CP-029, CP-046
RF-017	**Tutorial de bienvenida** · Prioridad **Baja** · Estado: **No verificable**
Descripción	La app muestra un tutorial inicial y recuerda que fue visto.
Entradas	Primer uso de la aplicación.
Proceso	Si `onboardingVisto` es falso se muestra `OnboardingView`; al terminar se marca como visto.
Salidas	El tutorial no vuelve a mostrarse.
Criterio de aceptación	Tras marcarlo como visto no vuelve a mostrarse.
Origen en el código	`OnboardingView`, `MarcarOnboardingVistoUseCase`
Historia y pruebas	HU-009; CP-025
RF-023	**Restablecimiento de datos** · Prioridad **Baja** · Estado: **No verificable**
Descripción	El administrador puede restablecer los datos de la aplicación.
Entradas	Confirmación del administrador.
Proceso	`ResetDataUseCase` restablece la configuración de la cuenta a sus valores iniciales.
Salidas	Datos de configuración restaurados.
Criterio de aceptación	La acción restablece los datos de la cuenta a su estado inicial.
Origen en el código	`ResetDataUseCase`, `MainViewModel.resetData`
Historia y pruebas	HU-009; CP-029

12.1.6 Asistente de IA y atención al cliente

Tabla 31. Fichas de requisitos funcionales: asistente de ia y atención al cliente (RF-018 a RF-021)

Campo	Contenido
RF-018	**Asistente de chat con IA** · Prioridad **Alta** · Estado: **No verificable**
Descripción	El usuario conversa con un asistente que responde con Gemini usando como contexto sus servicios, clientes y perfil.
Entradas	Mensaje del usuario, historial reciente y contexto del usuario.
Proceso	`SendAiChatMessageUseCase` clasifica la intención, construye el contexto del sistema y llama a `GeminiAiChatRepository` con `gemini-2.5-flash` (temperatura 0,7; máximo 800 tokens; historial de 6 turnos).

Campo	Contenido
Salidas	Respuesta del modelo o mensaje de error sin simular una respuesta de IA.
Criterio de aceptación	Con consultas generales se envía el contexto del usuario a Gemini (gemini-2.5-flash) y se muestra la respuesta; ante falla se muestra un mensaje de error sin simular una respuesta de IA.
Origen en el código	`ChatScreen`, `SendAiChatMessageUseCase`, `GeminiAiChatRepository`
Historia y pruebas	HU-010; CP-038, CP-048
RF-019	**Clasificación de intención y escalamiento** · Prioridad **Alta** · Estado: **Parcialmente cumplido**
Descripción	El chat clasifica la intención por palabras clave y, ante queja, cancelación, problema, error de la app o solicitud de persona, crea una solicitud de atención con prioridad.
Entradas	Mensaje del cliente.
Proceso	Detecta por palabras clave intenciones de queja, cancelación, problema, error de la aplicación o solicitud de persona; determina prioridad (Normal, Importante, Urgente) y crea la solicitud.
Salidas	Solicitud en `solicitudes_atencion`, notificación, actividad y mensaje con radicado de 8 caracteres.
Criterio de aceptación	El escalamiento crea una SolicitudAtencion en solicitudes_atencion, notifica al administrador y devuelve un radicado de 8 caracteres.
Origen en el código	`SendAiChatMessageUseCase (clasificarIntencion, evaluarEscalamiento)`, `CrearSolicitudAtencionUseCase`
Historia y pruebas	HU-011; CP-018, CP-039
RF-020	**Gestión de solicitudes de atención (administrador)** · Prioridad **Alta** · Estado: **Cumplido**
Descripción	El administrador toma, responde y resuelve las solicitudes de atención.
Entradas	Solicitud seleccionada por el administrador y su respuesta o notas.
Proceso	Los casos de uso Tomar, Responder y Resolver actualizan estado, responsable, respuesta y fecha de resolución.
Salidas	Solicitud en estado En Atención o Resuelto.
Criterio de aceptación	Tomar cambia el estado a En Atención y registra al responsable; responder guarda la respuesta; resolver cambia a Resuelto con fecha de resolución.
Origen en el código	`Tomar/Responder/ResolverSolicitudAtencionUseCase`, `EscalamientosAdminSection`, `AttentionRequiredSection`
Historia y pruebas	HU-012; CP-018, CP-019, CP-020, CP-021, CP-041
RF-021	**Portal del cliente** · Prioridad **Alta** · Estado: **Parcialmente cumplido**
Descripción	El cliente crea solicitudes de servicio o atención, consulta sus órdenes y ve las respuestas del equipo.
Entradas	Datos del servicio o motivo de atención solicitado por el cliente.
Proceso	`ClientePortalScreen` ofrece el formulario de solicitud y la consulta de órdenes y respuestas; el cliente solo accede a lo propio.
Salidas	Solicitud creada y respuestas visibles.
Criterio de aceptación	El cliente solo ve sus propias solicitudes; puede abrir el chat para preguntar sobre una orden.
Origen en el código	`ClientePortalScreen`, `MainViewModel.solicitarServicioCliente/solicitarAtencionHumana`
Historia y pruebas	HU-013; CP-040

12.1.7 Integración con Make

Tabla 32. Ficha del requisito funcional RF-022: integración con make

Campo	Contenido
RF-022	**Notificación de eventos a Make** · Prioridad **Media** · Estado: **Cumplido**
Descripción	Al crear, actualizar o completar un servicio se envía un evento al webhook de Make; el administrador puede enviar un reporte manual y probar la conexión.
Entradas	Evento (creado, actualizado, completado) y datos del servicio y cliente.
Proceso	`OkHttpMakeWebhookRepository` construye un JSON y lo envía por HTTP POST con tiempos de espera de 15 s; si no hay URL configurada omite el envío.
Salidas	HTTP 2xx o error registrado; prueba de conexión con mensaje al usuario.
Criterio de aceptación	Crear servicio envía SERVICIO_CREADO; completarlo envía SERVICIO_COMPLETADO con el estado Completado.
Origen en el código	`OkHttpMakeWebhookRepository`, `MakeWebhookRepository`, `MainViewModel.sendMakeReport/testMakeConnection`
Historia y pruebas	HU-014; CP-022, CP-023, CP-024, CP-043

12.2 Requisitos no funcionales

Los requisitos no funcionales se clasifican según las características del modelo ISO/IEC 25010. Los identificadores RNF-001 a RNF-012 coinciden con la matriz maestra; RNF-013 y siguientes completan las categorías de rendimiento y disponibilidad con comportamientos observables en el código.

12.2.1 Seguridad

Tabla 33. Requisitos no funcionales de seguridad

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-001	Las reglas de Firestore aplican mínimo privilegio: cada usuario accede a su árbol usuarios/{uid}; el rol no cambia tras el registro; cualquier otra ruta se deniega.	firestore.rules sin '|| true' ni comodines abiertos; desplegadas en servix-453fb.	Inspección de firestore.rules y despliegue con Firebase CLI. Sin pruebas con el emulador de reglas (CP-042).	Parcialmente cumplido
RNF-002	Ninguna clave de IA se empaqueta en el APK: el chat usa Firebase AI Logic.	No existe GEMINI_API_KEY en BuildConfig ni en el código.	Revisión de código y hallazgo H-02 corregido.	Cumplido
RNF-003	Los atajos de acceso de prueba solo existen en builds debug.	El bloque de atajos está condicionado a BuildConfig.DEBUG.	Revisión de AuthScreen.kt (H-04).	Cumplido

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-004	El repositorio no contiene secretos: `.env` y `local.properties` están fuera de control de versiones, `.env.example` solo documenta la ausencia de variables obligatorias y la URL del webhook de Make se inyecta al compilar (`make.webhook.url` o `MAKE_WEBHOOK_URL`).	Sin credenciales ni URL de webhook en el repositorio.	Búsqueda en el repositorio; la URL compilada sigue siendo extraíble del APK (riesgo R-04).	Cumplido

12.2.2 Rendimiento

Tabla 34. Requisitos no funcionales de rendimiento

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-013	La interfaz muestra el progreso mientras espera la respuesta del asistente y evita enviar un segundo mensaje durante la espera.	`isChatLoading` bloquea el reenvío; el chat muestra un indicador de carga.	Revisión de `MainViewModel.sendChatMessage` (sin medición de tiempos).	Parcialmente cumplido
RNF-014	Las solicitudes a la IA acotan el historial y la longitud de la respuesta para controlar latencia y costo.	Historial de 6 turnos y `maxOutputTokens = 800`.	Revisión de `GeminiAiChatRepository`.	Cumplido
RNF-015	Los datos se actualizan en tiempo real mediante escuchas de Firestore, sin recarga manual.	Cada repositorio Firebase usa `addSnapshotListener` por usuario.	Revisión de código; sin medición de latencia.	Parcialmente cumplido

12.2.3 Usabilidad

Tabla 35. Requisitos no funcionales de usabilidad

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-007	Interfaz Material 3 con identidad azul de ServiX, modo claro y oscuro y estados vacíos y de error comprensibles.	Paleta definida en core/theme/Color.kt; modo oscuro configurable.	Revisión de código; sin verificación visual en dispositivo (CP-046).	Parcialmente cumplido
RNF-019	Los controles principales de la interfaz exponen etiquetas de prueba (`testTag`) y descripciones de contenido para pruebas de UI.	Barra inferior con `testTag` por destino; pruebas Robolectric de navegación.	Pruebas CP-025 a CP-029.	Cumplido

12.2.4 Mantenibilidad

Tabla 36. Requisitos no funcionales de mantenibilidad

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-006	Arquitectura en capas (presentación, dominio, datos, núcleo) con repositorios definidos por interfaz y DI manual en AppContainer.	Las capas dependen del dominio; los repositorios se sustituyen por dobles en pruebas.	Revisión de la estructura de paquetes y pruebas con repositorios simulados.	Cumplido
RNF-009	El proyecto cuenta con pruebas unitarias y de UI sobre JVM ejecutables con Gradle.	gradlew testDebugUnitTest finaliza sin fallos.	29 pruebas, 0 fallos (31/10/2026 local; 01/11/2026 Azure Pipelines).	Cumplido
RNF-012	Los componentes de presentación se concentran de forma razonable; el MainViewModel no debe crecer sin control.	MainViewModel.kt de unos 25 KB (candidato a dividirse).	Observación del tamaño del archivo.	Parcialmente cumplido

12.2.5 Portabilidad

Tabla 37. Requisitos no funcionales de portabilidad

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-005	La app es compatible con Android 8.0 (API 26) o superior y apunta a la API 36.	minSdk 26 y targetSdk 36 en app/build.gradle.kts.	Lectura de la configuración; compilación debug exitosa.	Cumplido
RNF-010	El proyecto compila con el wrapper de Gradle en un equipo limpio y en Azure Pipelines.	assembleDebug exitoso local y en el pipeline ServiX-CI.	Run 20261101.3 exitoso con artefacto servix-apk-debug.	Cumplido
RNF-018	El sistema compila sin cambios en Windows (equipo local) y en Linux (agente `ubuntu-latest`) usando el wrapper de Gradle.	`gradlew assembleDebug` exitoso en ambos entornos.	Compilación local del 31/10/2026 y run 20261101.3 de Azure Pipelines.	Cumplido

12.2.6 Disponibilidad

Tabla 38. Requisitos no funcionales de disponibilidad

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-008	Si la IA falla, el sistema informa el error y no presenta respuestas predefinidas como si fueran de IA.	El caso de uso devuelve un mensaje de error y sugiere reintentar o pedir un asesor.	Revisión de SendAiChatMessageUseCase (H-03); sin prueba automática (CP-048).	Parcialmente cumplido
RNF-011	Los datos se almacenan en Cloud Firestore con escuchas en tiempo real por usuario.	Los repositorios Firebase usan addSnapshotListener sobre usuarios/{uid}/...	Revisión de código; sin ejecución contra Firestore real (CP-035).	Parcialmente cumplido
RNF-016	El envío de eventos a Make usa tiempos de espera de 15 s y no bloquea la interfaz.	Conexión y lectura con 15 s; envío en corrutina de E/S.	Revisión de `OkHttpMakeWebhookRepository`.	Cumplido

ID	Requisito	Criterio de aceptación	Verificación	Estado
RNF-017	La aplicación tolera fallos de Firebase en el arranque pasando a repositorios simulados.	`AppContainer` y `MainActivity` capturan excepciones y activan los repositorios mock.	Revisión de código. Es una degradación silenciosa (hallazgo H-11, Bug 46).	Parcialmente cumplido

12.3 Interfaces externas

12.3.1 Interfaces de usuario

La interfaz sigue Material 3 con la paleta de ServiX (azul oscuro `#091E42`, azul `#0066F5`, azul claro `#00A3FF`, fondo claro `#E0F2FE` y acento ámbar `#F59E0B`), tema claro y oscuro y diseño adaptable. La navegación se define en `AppRoutes` y `BottomNavItem`.

Tabla 39. Destinos de navegación de la aplicación

Ruta	Pantalla	Rol	Función
`auth`	Autenticación	Anónimo	Registro, inicio de sesión y recuperación de contraseña.
`home`	Inicio	Administrador	Estadísticas, metas, atención requerida y búsqueda global.
`clientes`, `cliente_detalle/{id}`	Clientes	Administrador	Lista, alta, edición y detalle de clientes.
`servicios`, `servicio_detalle/{id}`	Servicios	Administrador	Lista, alta, edición, estados y detalle.
`historial`	Historial	Administrador	Actividades y centro financiero.
`chat`	Chat IA	Cliente y Administrador	Asistente conversacional y solicitudes de atención.
`perfil`	Perfil	Administrador	Perfil del negocio, ajustes y prueba de Make.
`cliente_portal`	Portal del cliente	Cliente	Solicitudes, órdenes y respuestas del equipo.

12.3.2 Interfaces de software

Tabla 40. Interfaces de software

Sistema	Propósito	Mecanismo	Datos intercambiados
Firebase Authentication	Identidad de los usuarios.	SDK `firebase-auth`; correo y contraseña; `AuthStateListener`.	Credenciales, uid, nombre visible.
Cloud Firestore	Persistencia en tiempo real.	SDK `firebase-firestore`; escuchas y escrituras por documento.	Colecciones descritas en la sección 12.5.

Sistema	Propósito	Mecanismo	Datos intercambiados
Firebase AI Logic	Respuestas del asistente.	SDK `firebase-ai`; `Firebase.ai(backend = googleAI())`; modelo `gemini-2.5-flash`.	Mensaje, historial y contexto del sistema; texto de respuesta.
Make (opcional)	Automatización de avisos y reportes.	HTTP POST JSON con OkHttp hacia la URL configurada al compilar.	Evento y datos del servicio y del cliente.
Azure DevOps	Gestión y CI del proyecto.	Azure Repos, Boards y Pipelines (no interviene en la ejecución).	Código, tareas y resultados de pruebas.

12.3.3 Interfaces de hardware y comunicaciones

- Hardware: dispositivo Android con pantalla táctil; cámara o galería para la foto de perfil o de cliente (se almacena en Base64).
- Comunicaciones: HTTPS hacia los servicios de Firebase y hacia la URL de Make; no se abren puertos ni se usan protocolos propios.
- No se requieren sensores, GPS, Bluetooth ni permisos especiales más allá del acceso a Internet.

12.4 Requisitos de seguridad

Tabla 41. Requisitos de seguridad

ID	Requisito	Implementación	Estado
SEC-01	Mínimo privilegio en el almacenamiento: denegar todo lo no declarado.	`firestore.rules` termina con `match /{document=**}` y `allow read, write: if false`.	Cumplido (inspección)
SEC-02	Cada usuario solo accede a su árbol de datos.	`usuarios/{userId}` con `isOwner(userId) || isAdmin()` y lista cerrada de subcolecciones.	Cumplido (inspección)
SEC-03	El rol no puede cambiarse tras el registro.	Regla `update`: `request.resource.data.role == resource.data.role` para el dueño.	Cumplido (inspección)
SEC-04	Un cliente solo lee sus solicitudes de atención.	Regla: `isAdmin() || resource.data.clienteId == request.auth.uid`.	Cumplido (inspección)
SEC-05	Validación de tamaño en la creación de solicitudes.	Regla `create` con límites de longitud (cliente 128, nombre 200, motivo 500, mensaje 4000).	Cumplido (inspección)
SEC-06	Sin claves de IA en el cliente.	Firebase AI Logic; no existe `GEMINI_API_KEY` en el código.	Cumplido
SEC-07	Sin secretos en el repositorio.	`.gitignore` excluye `.env`, `local.properties`, `*.jks` y `*.keystore`.	Cumplido
SEC-08	Atajos de acceso solo en depuración.	Bloque condicionado por `BuildConfig.DEBUG` en `AuthScreen`; sin contraseñas incluidas.	Cumplido
SEC-09	El rol de administrador no debe poder autoasignarse.	No implementado: el registro permite elegir `ADMINISTRADOR`.	No cumplido (H-15)

ID	Requisito	Implementación	Estado
SEC-10	Limitar el uso de la API de IA al aplicativo legítimo (App Check).	La dependencia de App Check existe pero no se inicializa ningún proveedor.	No cumplido (R-05)
SEC-11	Restringir la clave de API del proyecto Firebase por paquete y huella.	Acción pendiente en la consola de Google Cloud.	Pendiente (R-06)

Nota. Reglas sin pruebas del emulador
Las reglas de Firestore están desplegadas y fueron revisadas por inspección, pero no se probaron con el emulador de reglas (caso CP-042, no ejecutado). Hasta entonces, SEC-01 a SEC-05 se consideran verificados solo estáticamente.

12.5 Modelo de datos

Los datos se organizan en un modelo multiusuario: cada usuario posee un árbol bajo `usuarios/{uid}` con sus clientes, servicios, notificaciones, actividades y configuración, y existe una colección raíz independiente para las solicitudes de atención. La documentación completa con tipos y ejemplos se encuentra en DOC-07.

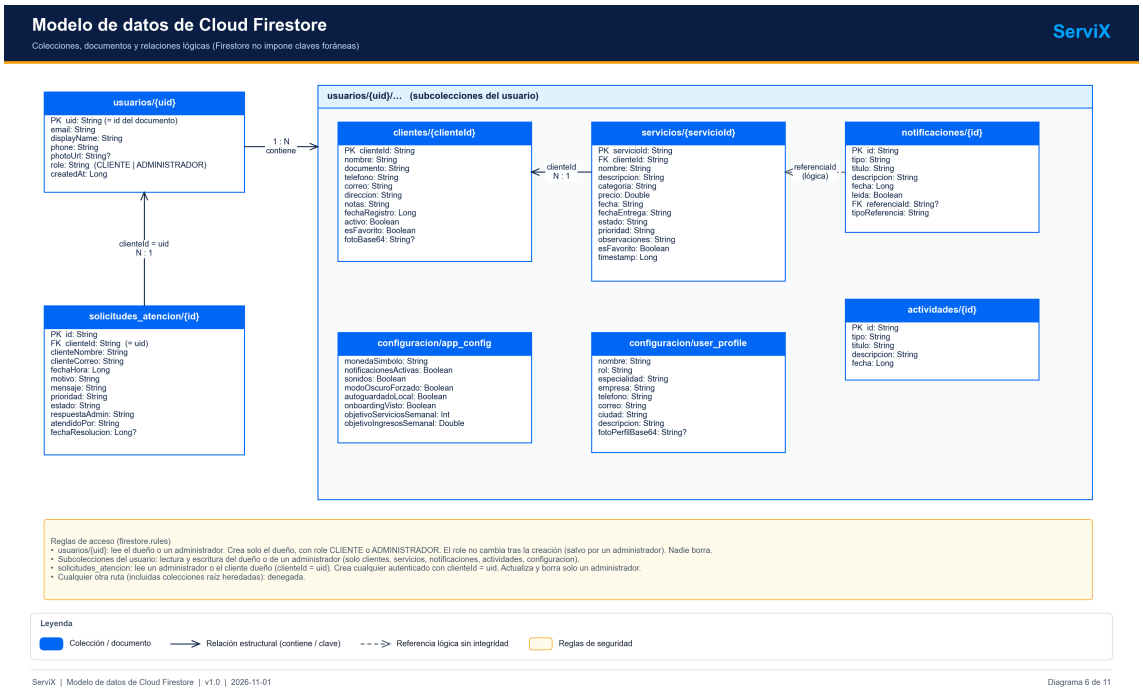


Figura 10. Modelo de datos de ServiX en Cloud Firestore
Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 42. Colecciones de Firestore

Ruta	Contenido	Clave del documento
`usuarios/{uid}`	Perfil de la cuenta: correo, nombre, teléfono, foto, rol y fecha de creación.	uid de Firebase Auth
`usuarios/{uid}/clientes/{id}`	Clientes del negocio.	Identificador generado
`usuarios/{uid}/servicios/{id}`	Órdenes de servicio vinculadas a un cliente.	Identificador generado

Ruta	Contenido	Clave del documento
`usuarios/{uid}/notificaciones/{id}`	Notificaciones internas.	UUID
`usuarios/{uid}/actividades/{id}`	Historial de actividades.	Identificador generado
`usuarios/{uid}/configuracion/{app_config | user_profile}`	Ajustes de la aplicación y perfil del negocio.	Nombre fijo
`solicitudes_atencion/{id}`	Casos escalados desde el chat.	UUID

12.6 Casos de uso

Los casos de uso resumen la interacción de los actores con el sistema. El diagrama general se complementa con descripciones textuales de los casos más relevantes.

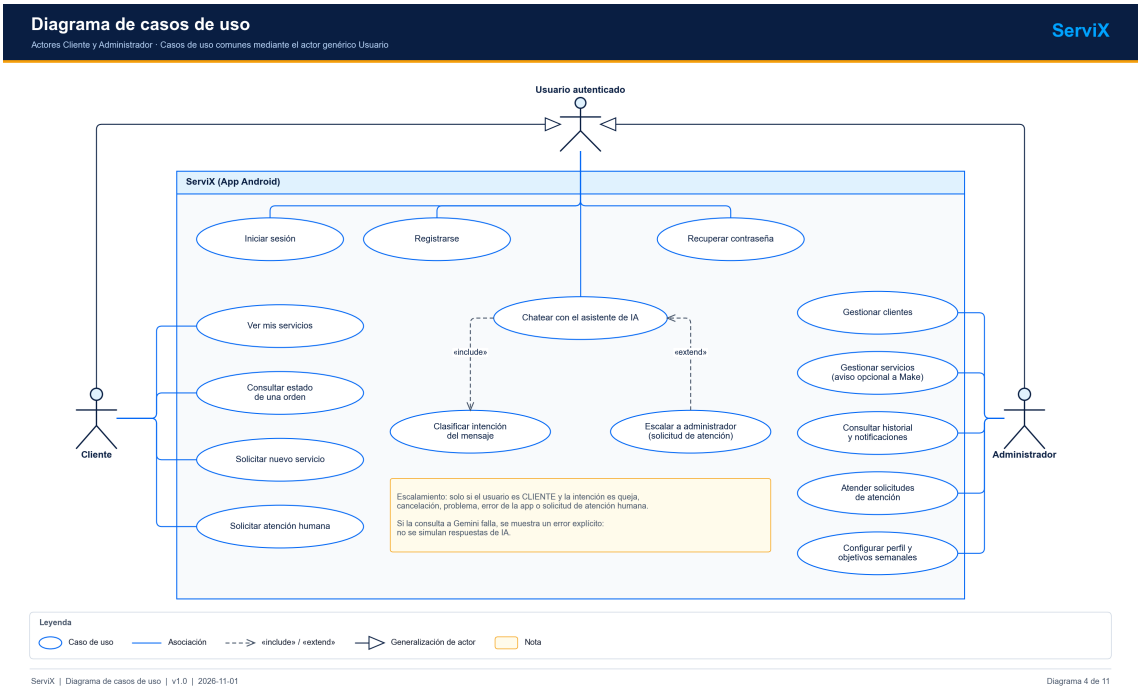


Figura 11. Diagrama de casos de uso de ServiX

Fuente: Diagrama del Equipo ServiX (04_Arquitectura_Diagramas).

Tabla 43. Descripción de los casos de uso

Caso	Actor y precondición	Flujo principal	Alternos y excepciones	Requisitos
CU-01 Registrarse e iniciar sesión	Cliente o Administrador. Pantalla de autenticación visible.	1) El actor elige su rol. 2) Ingresa los datos. 3) El sistema valida los campos. 4) Firebase autentica. 5) El sistema lee el rol almacenado y navega.	Contraseña corta o correo inválido: se muestra el error. Rol distinto al almacenado: se rechaza el acceso.	RF-001 a RF-005
CU-02 Gestionar clientes	Administrador. Sesión de administrador.	1) Abre Clientes. 2) Crea o edita un cliente. 3) El sistema valida y guarda. 4) Se registra la actividad y se notifica.	Eliminar un cliente borra sus servicios; el actor puede deshacer desde el aviso.	RF-006 a RF-009

Caso	Actor y precondición	Flujo principal	Alternos y excepciones	Requisitos
CU-03 Gestionar servicios	Administrador. Existe al menos un cliente.	1) Abre Servicios y crea una orden. 2) Cambia el estado según avance. 3) El sistema registra actividad y notifica a Make al crear o completar.	Sin URL de Make configurada, el envío se omite sin afectar la operación.	RF-010, RF-011, RF-022
CU-04 Consultar historial y notificaciones	Administrador. Sesión de administrador.	1) Abre Historial o el centro de notificaciones. 2) Revisa y marca como leídas o elimina.	Sin datos: se muestra un estado vacío.	RF-012, RF-013
CU-05 Conversar con el asistente de IA	Cliente o Administrador. Sesión iniciada.	1) Escribe un mensaje. 2) El sistema clasifica la intención. 3) Si no requiere escalamiento, consulta a Gemini con el contexto. 4) Muestra la respuesta.	Falla de la IA: se muestra un mensaje de error genérico y se sugiere reintentar o pedir un asesor.	RF-018
CU-06 Escalar a atención humana	Cliente. Sesión de cliente.	1) El mensaje indica queja, cancelación, problema, error o solicitud de persona. 2) El sistema crea la solicitud con prioridad. 3) Registra actividad y notificación. 4) Informa el radicado.	Mensajes sin esas intenciones continúan con la IA.	RF-019
CU-07 Atender solicitudes	Administrador. Existen solicitudes pendientes.	1) Abre la sección de atención. 2) Toma la solicitud. 3) Responde. 4) La resuelve.	Solo el administrador puede modificar el estado (regla).	RF-020
CU-08 Solicitar servicios desde el portal	Cliente. Sesión de cliente.	1) Abre el Portal. 2) Completa el formulario. 3) Consulta sus órdenes y las respuestas del equipo.	Solo ve sus propias solicitudes.	RF-021
CU-09 Configurar el negocio	Administrador. Sesión de administrador.	1) Abre Perfil. 2) Edita perfil, metas y ajustes. 3) Prueba la conexión con Make si procede.	Restablecer datos exige confirmación.	RF-015 a RF-017, RF-022, RF-023

12.7 Matriz de trazabilidad resumida

La matriz vincula cada requisito con su historia de usuario, los casos de prueba y su estado. El detalle completo, incluidas las evidencias, se mantiene en las hojas de cálculo del proyecto.

Tabla 44. Trazabilidad resumida por módulo: requisitos, historias y casos de prueba

Módulo	Requisitos	Historias	Casos de prueba	Estado
Autenticación y roles	RF-001 a RF-005	HU-001, HU-002, HU-003	CP-003, CP-004, CP-014, CP-015, CP-016, CP-017, CP-025, CP-026, CP-027, CP-028, CP-029, CP-030, CP-031, CP-032, CP-033, CP-034	3 parcialmente cumplido; 1 cumplido; 1 no verificable
Gestión de clientes	RF-006 a RF-009	HU-004, HU-005	CP-005, CP-006, CP-007, CP-009, CP-011, CP-012, CP-035, CP-036	2 cumplido; 2 parcialmente cumplido
Gestión de servicios	RF-010 a RF-011	HU-006	CP-005, CP-008, CP-010, CP-012, CP-023, CP-036	2 cumplido
Historial, notificaciones y búsqueda	RF-012 a RF-014	HU-007, HU-008	CP-018, CP-028, CP-037	2 parcialmente cumplido; 1 no verificable
Panel de inicio, perfil y configuración	RF-015 a RF-023	HU-008, HU-009	CP-025, CP-029, CP-046	2 parcialmente cumplido; 2 no verificable

Módulo	Requisitos	Historias	Casos de prueba	Estado
Asistente de IA y atención al cliente	RF-018 a RF-021	HU-010, HU-011, HU-012, HU-013	CP-018, CP-019, CP-020, CP-021, CP-038, CP-039, CP-040, CP-041, CP-048	1 no verificable; 2 parcialmente cumplido; 1 cumplido
Integración con Make	RF-022	HU-014	CP-022, CP-023, CP-024, CP-043	1 cumplido

La trazabilidad requisito por requisito se encuentra en las fichas de la sección 12.1 y en el Anexo A.

12.8 Criterios de aceptación y verificación

Tabla 45. Métodos de verificación aplicados

Método	Descripción	Evidencia disponible
Inspección de código	Lectura del código y de las reglas para confirmar el comportamiento.	Columna de origen de cada ficha.
Prueba unitaria (JVM)	Pruebas JUnit sobre casos de uso y repositorios simulados.	Clases `ServixUnitTests`, `ServixAuthUnitTests`, `ServixAtencionUnitTests`, `ServixRepositoryTest`.
Prueba de integración con doble	Casos de uso con un webhook simulado.	`ServixMakeIntegrationTest`.
Prueba de UI con Robolectric	Lanzamiento de la aplicación y navegación sin dispositivo.	`ServixRobolectricTest`, `ExampleRobolectricTest`, `GreetingScreenshotTest`.
Compilación y CI	Compilación del APK y pruebas en cada cambio.	Run 20261101.3 de Azure Pipelines (100 % de pruebas aprobadas).
Prueba en dispositivo o con servicios reales	Autenticación, Firestore, Gemini, Make y reglas.	**No ejecutada** (CP-030 a CP-048 pendientes).

Resultado global: de los 66 casos de prueba definidos, 47 se han ejecutado y aprobado (29 pruebas automatizadas, una compilación local, una ejecución del pipeline y 16 renders de pantallas con datos simulados) y 19 permanecen sin ejecutar por requerir dispositivo, emulador o servicios externos. Ningún caso ejecutado ha fallado en las ejecuciones finales.

> **Advertencia. Alcance de la verificación**
> La aprobación de las pruebas automatizadas confirma la lógica de dominio y la navegación con datos simulados. No demuestra el funcionamiento contra Firebase y Gemini reales, que figura como pendiente.

13 Resultados verificables y limitaciones

Esta sección reúne los resultados que pueden comprobarse con evidencia y declara con la misma claridad lo que no se ha verificado.

13.1 Resultados obtenidos

Tabla 46. Resultados verificables del proyecto

Resultado	Valor	Evidencia
Pruebas automatizadas aprobadas (ejecución local, 31/10/2026)	29 de 29	Informes JUnit; `Resultados_JUnit.csv`.
Pruebas automatizadas aprobadas (Azure Pipelines, 01/11/2026)	100 % (run 20261101.3)	Captura del run y pestaña de pruebas.
Compilación del APK de depuración (local)	BUILD SUCCESSFUL en 20 min 21 s	Registro de Gradle.
Duración del run exitoso de CI	9 min 42 s	Captura del run.
Artefacto publicado	`servix-apk-debug`	Pipeline ServiX-CI.
Requisitos funcionales especificados	23	Sección 12.1.
Casos de prueba definidos / ejecutados y aprobados	66 / 47	DOC-04 y `Casos_Prueba.xlsx`.
Hallazgos de auditoría registrados	20 (la mayoría corregidos)	Anexo C.
Fallas de CI corregidas	2 (runs 1 y 2)	DOC-04, sección de defectos.

13.2 Estado de los requisitos

Tabla 47. Estado de verificación de los requisitos funcionales

Estado	Cantidad	Porcentaje
Parcialmente cumplido	11	48 %
Cumplido	7	30 %
No verificable	5	22 %
Total	**23**	100 %

13.3 Limitaciones honestas

- **No se ejecutaron** pruebas en dispositivo ni en emulador: el inicio de sesión por rol, la gestión de datos sobre Firestore y la interfaz real no están verificados.
- **No se ejecutó** ninguna prueba contra Firebase y Gemini reales; la comunicación con el modelo es un resultado pendiente.
- **No se ejecutó** la prueba real de Make, Android Lint, las pruebas instrumentadas ni las pruebas de las reglas de Firestore.
- Las capturas de la aplicación de este documento son renders de Robolectric con datos de demostración, no capturas de un dispositivo.
- El rol Administrador puede autoasignarse en el registro (H-15) y la degradación a repositorios simulados es silenciosa (H-11).
- No hay autorización ni política de tratamiento de datos personales, ni archivo LICENSE.

- Las fechas de los Sprints son provisionales y dos invitaciones de Azure DevOps siguen pendientes.

14 Viabilidad

14.1 Viabilidad técnica

El proyecto es técnicamente viable: se construyó con tecnologías estándar de Android y servicios gestionados, compila de forma reproducible en un entorno limpio (Azure Pipelines) y sus pruebas automatizadas pasan. Los riesgos técnicos identificados (autoasignación de rol, degradación silenciosa, ausencia de App Check) tienen soluciones conocidas y acotadas. La principal restricción es el equipo de desarrollo de 4 GB de RAM, mitigada mediante integración continua.

14.2 Viabilidad operativa

La aplicación no requiere servidores propios, lo que reduce la carga de operación a la administración del proyecto Firebase (reglas, cuotas y costos) y a la configuración opcional de Make. Un negocio pequeño puede adoptarla con poca capacitación, dado que la interfaz sigue patrones habituales de Material 3. Para operar con datos reales se requiere antes resolver las brechas de protección de datos y completar las pruebas en dispositivo.

14.3 Viabilidad económica

No se han inventado cifras de esfuerzo ni de ingresos. La tabla resume los costos observables y lo que debe verificarse antes de operar.

Tabla 48. Costos y recursos del proyecto

Elemento	Costo o condición	Observación
Herramientas de desarrollo (Android Studio, Visual Studio Code, Git)	Gratuitas	Uso permitido para estudiantes.
Azure DevOps (Boards, Repos, Pipelines)	Nivel gratuito para equipos pequeños	Los agentes alojados requieren que la organización disponga de paralelismo gratuito concedido.
GitHub (repositorio público)	Gratuito	Repositorio https://github.com/JuanXD2201/ServiX.
Firebase (Authentication, Firestore, AI Logic)	Depende del plan del proyecto y del uso	El plan de facturación no se documenta en el repositorio; verificar cuotas y alertas.
API de Gemini	Según cuota y plan	Configurar límites y App Check para controlar costos.
Make	Según plan de la cuenta	Opcional; revisar los límites del plan.
Equipo de cómputo e Internet	Existentes	Equipo de referencia con 4 GB de RAM.
Publicación en una tienda	No aplica	No se publica; una cuenta de desarrollador de Google Play tiene un costo si se decidiera publicar.

En conclusión, el costo directo registrado del desarrollo es bajo; el costo de operación depende del volumen de uso de Firebase y Gemini y debe medirse con cuotas y alertas.

15 Conclusiones, recomendaciones y trabajo futuro

15.1 Conclusiones

- ServiX cumple, con datos simulados, los objetivos de gestión de clientes, servicios y atención, con 29 de 29 pruebas automatizadas aprobadas en dos entornos.
- La arquitectura por capas y los repositorios por interfaz permitieron probar la lógica en la JVM y automatizar la integración continua.
- La documentación basada en el código y en evidencias reales permite distinguir lo verificado de lo pendiente.
- El producto no está listo para operar con datos reales: faltan pruebas en dispositivo y con servicios reales, y deben cerrarse brechas de seguridad y de protección de datos personales.
- Las fallas de CI (dependencias sin `:app` y JDK 17 con SDK 36) enseñaron a validar el entorno y a alinear las versiones de Java.

15.2 Recomendaciones

Tabla 49. Recomendaciones priorizadas

Prioridad	Recomendación
Alta	Asignar el rol de administrador desde un entorno de confianza y quitar la opción del registro.
Alta	Ejecutar las pruebas en dispositivo y con Firebase y Gemini reales; probar las reglas de Firestore.
Alta	Redactar la política de tratamiento de datos y registrar la autorización de los titulares.
Media	Registrar App Check, restringir la clave de API y configurar cuotas y alertas.
Media	Añadir un archivo LICENSE y el aviso de licencias de terceros.
Media	Mostrar un aviso cuando se degrade a repositorios simulados.
Baja	Dividir `MainViewModel`, implementar o retirar Retell y n8n.

15.3 Trabajo futuro

- Notificaciones push con Firebase Cloud Messaging.
- Destino común para las notificaciones de atención dirigidas al administrador.
- Asignación de técnicos y programación de rutas.
- Versión para otras plataformas si el producto se orienta a un mercado más amplio.
- Estudio con usuarios reales para validar la usabilidad y la propuesta de valor.

Referencias

Referencias en formato APA. Para las fuentes en línea se indica el sitio de origen y el título; la fecha de consulta y la versión vigente deben verificarse al revisar este documento.

ACM. (2018). *ACM code of ethics and professional conduct*. Association for Computing Machinery. acm.org

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems, 33*.

Comunidad Andina. (1993). *Decisión 351: Régimen común sobre derecho de autor y derechos conexos*. Comisión del Acuerdo de Cartagena.

Congreso de la República de Colombia. (1982). *Ley 23 de 1982: Sobre derechos de autor*.

Congreso de la República de Colombia. (1999). *Ley 527 de 1999: Por medio de la cual se define y reglamenta el acceso y uso de los mensajes de datos, del comercio electrónico y de las firmas digitales*.

Congreso de la República de Colombia. (2008). *Ley 1266 de 2008: Por la cual se dictan las disposiciones generales del hábeas data*.

Congreso de la República de Colombia. (2009). *Ley 1273 de 2009: Por medio de la cual se modifica el Código Penal, se crea un nuevo bien jurídico tutelado denominado de la protección de la información y de los datos*.

Congreso de la República de Colombia. (2011). *Ley 1480 de 2011: Estatuto del Consumidor*.

Congreso de la República de Colombia. (2012). *Ley 1581 de 2012: Por la cual se dictan disposiciones generales para la protección de datos personales*.

Congreso de la República de Colombia. (2018). *Ley 1915 de 2018: Por la cual se modifica la Ley 23 de 1982 y se establecen otras disposiciones en materia de derecho de autor y derechos conexos*.

Google. (s. f.-a). *Guide to app architecture*. Android Developers. developer.android.com

Google. (s. f.-b). *Cloud Firestore documentation*. Firebase. firebase.google.com

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

IEEE. (1998). *IEEE Std 830-1998: IEEE recommended practice for software requirements specifications*. IEEE.

IEEE. (2009). *IEEE Std 1016-2009: IEEE standard for information technology — Systems design — Software design descriptions*. IEEE.

ISO/IEC. (2011). *ISO/IEC 25010:2011: Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models*. ISO.

ISO/IEC. (2022a). *ISO/IEC 27001:2022: Information security, cybersecurity and privacy protection — Information security management systems — Requirements*. ISO.

ISO/IEC. (2022b). *ISO/IEC 27002:2022: Information security, cybersecurity and privacy protection — Information security controls*. ISO.

ISO/IEC/IEEE. (2013). *ISO/IEC/IEEE 29119: Software and systems engineering — Software testing*. ISO.

ISO/IEC/IEEE. (2018). *ISO/IEC/IEEE 29148:2018: Systems and software engineering — Life cycle processes — Requirements engineering*. ISO.

ISO/IEC/IEEE. (2022). *ISO/IEC/IEEE 42010:2022: Software, systems and enterprise — Architecture description*. ISO.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps handbook*. IT Revolution Press.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

OWASP Foundation. (2023). *OWASP mobile application security verification standard (MASVS)*. owasp.org

Presidencia de la República de Colombia. (2013). *Decreto 1377 de 2013: Por el cual se reglamenta parcialmente la Ley 1581 de 2012*.

Presidencia de la República de Colombia. (2015). *Decreto 1074 de 2015: Decreto Único Reglamentario del Sector Comercio, Industria y Turismo*.

Schwaber, K., & Sutherland, J. (2020). *The Scrum guide*. scrumguides.org

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*.

Anexo A. Matriz de trazabilidad de historias de usuario

Tabla 50. Historias de usuario, requisitos y elementos del backlog

HU	Rol	Quiero	Requisitos	PBI en Azure	Estado
HU-001	Cliente / Administrador	registrarme con mi correo y contraseña	RF-001	Autenticación con roles Cliente y Administrador (PBI 10)	Parcialmente cumplido
HU-002	Cliente / Administrador	iniciar y cerrar sesión con mi rol	RF-002, RF-003, RF-005	Autenticación con roles Cliente y Administrador (PBI 10)	Parcialmente cumplido
HU-003	Cliente / Administrador	recuperar mi contraseña por correo	RF-004	Autenticación con roles Cliente y Administrador (PBI 10)	No verificable
HU-004	Administrador	registrar, editar, eliminar y restaurar clientes	RF-006, RF-007, RF-009	Gestión de clientes (alta, edición, borrado, restauración) (PBI 14)	Cumplido
HU-005	Administrador	marcar clientes y servicios como favoritos	RF-008	Gestión de clientes (PBI 14) / Gestión de servicios (PBI 18)	Parcialmente cumplido
HU-006	Administrador	crear servicios y cambiar su estado	RF-010, RF-011	Gestión de servicios y seguimiento de estados (PBI 18)	Cumplido
HU-007	Administrador	consultar el historial y recibir notificaciones	RF-012, RF-013	Historial y notificaciones	Parcialmente cumplido
HU-008	Administrador	buscar en todo el negocio y ver mis metas semanales	RF-014, RF-015	Gestión de clientes / servicios (PBI 14, 18)	Parcialmente cumplido
HU-009	Administrador	editar el perfil del negocio y la configuración	RF-016, RF-017, RF-023	Gestión de servicios y seguimiento de estados (PBI 18)	Parcialmente cumplido
HU-010	Cliente / Administrador	conversar con un asistente de IA	RF-018	Chat con IA y escalamiento a administrador	No verificable
HU-011	Cliente	que el chat escale mi queja o problema a una persona	RF-019	Chat con IA y escalamiento a administrador	Parcialmente cumplido
HU-012	Administrador	tomar, responder y resolver solicitudes de atención	RF-020	Chat con IA y escalamiento a administrador	Cumplido
HU-013	Cliente	pedir servicios y ver mis solicitudes en un portal	RF-021	Chat con IA y escalamiento a administrador	Parcialmente cumplido
HU-014	Administrador	que los eventos de servicio lleguen a Make	RF-022	Gestión de servicios y seguimiento de estados (PBI 18)	Cumplido

HU	Rol	Quiero	Requisitos	PBI en Azure	Estado
HU-015	Equipo de desarrollo	contar con integración continua que compile y pruebe	RNF-009, RNF-010	Pipeline CI/CD en Azure Pipelines (PBI 32)	Cumplido

Anexo B. Registro de riesgos

Tabla 51. Riesgos identificados

ID	Riesgo	Categoría	P	I	Mitigación	Estado
R-01	El registro permite elegir el rol ADMINISTRADOR (autoasignación de privilegios).	Seguridad	4	5	Asignar roles con claims de administrador (Admin SDK o Cloud Function) y quitar la opción del registro.	Abierto
R-02	AppContainer degrada a repositorios simulados sin avisar al usuario si falla Firebase.	Fiabilidad	3	4	Mostrar un aviso visible y bloquear la operación cuando no haya Firebase.	Abierto
R-03	Interfaces RetellAIRepository y N8nWebhookRepository sin implementación.	Alcance	2	2	Decidir si se implementan o se eliminan; documentarlas como trabajo futuro.	Abierto
R-04	La URL del webhook de Make se compila dentro del APK y puede extraerse de él (ya no está en el código fuente).	Seguridad	4	3	Mover la llamada a un backend o función en la nube con la URL como secreto.	Mitigado pa rcialmente
R-05	App Check no está registrado: el uso de Gemini no se limita al APK legítimo.	Seguridad / Costos	3	3	Registrar App Check (por ejemplo Play Integrity) e inicializar el proveedor en la app.	Abierto
R-06	La clave de API de google-services.json no está restringida por paquete y SHA-1.	Seguridad	3	4	Restringir la clave en Google Cloud Console (acción del propietario).	Abierto
R-07	No hay pruebas en dispositivo o emulador: el comportamiento real contra Firebase y Gemini no está verificado.	Calidad	4	4	Instalar el APK en un dispositivo físico y ejecutar los casos CP-030 a CP-048.	Abierto
R-08	Las fechas de los Sprints en Azure DevOps son provisionales.	Gestión	3	2	Confirmar las fechas con el calendario académico y ajustarlas en Azure.	Abierto
R-09	Dos integrantes fueron invitados a Azure DevOps y aún no aceptan la invitación.	Gestión	3	3	Pedir la aceptación y verificar que puedan actualizar elementos del tablero.	Abierto
R-10	Compilaciones lentas por el equipo local (4 GB de RAM; assembleDebug tardó 20 min 21 s).	Productividad	4	2	Apoyarse en Azure Pipelines para compilar y probar.	Mitigado
R-11	Costos y cuotas del uso de la API de Gemini (Developer API) sin límites configurados.	Costos	2	3	Configurar cuotas, App Check y monitoreo en la consola de Firebase.	Abierto
R-12	El pipeline usa la etiqueta ubuntu-latest, que migrará de versión (advertencia del propio Azure).	Mantenibilidad	3	2	Fijar una versión de imagen estable y revisar las advertencias del pipeline.	Abierto
R-13	MainViewModel concentra gran parte del estado de la aplicación (cerca de 25 KB).	Mantenibilidad	3	3	Dividir por funcionalidad (autenticación, clientes, servicios, chat) sin cambiar el comportamiento.	Abierto
R-14	Una clave de Gemini antigua sigue en el .env local aunque ya no se usa.	Seguridad	2	4	Revocar la clave en Google AI Studio (acción del propietario).	Abierto
R-15	Android Lint y pruebas instrumentadas no se han ejecutado.	Calidad	3	3	Ejecutar lintDebug y connectedDebugAndroidTest cuando haya dispositivo.	Abierto

ID	Riesgo	Categoría	P	I	Mitigación	Estado
R-16	Valores por defecto inconsistentes: SignUpUseCase y AuthUser usan ADMINISTRADOR por defecto, contrario al mínimo privilegio de H-17.	Seguridad	3	4	Corregido el 01/11/2026: SignUpUseCase y AuthUser usan CLIENTE por defecto (commit 605a91d).	Cerrado

P = probabilidad e I = impacto, en escala de 1 a 5.

Anexo C. Hallazgos de la auditoría técnica (H-01 a H-21)

Tabla 52. Hallazgos de la auditoría y su estado

ID	Prioridad	Hallazgo	Estado	Acción o evidencia
H-01	P0	Reglas de Firestore abiertas (|| true en todas las reglas y comodín /{document=**}).	Corregido	Reglas de mínimo privilegio desplegadas en servix-453fb.
H-02	P0	GEMINI_API_KEY empaquetada en el APK (BuildConfig) y enviada en la URL.	Corregido	Migración a Firebase AI Logic; sin clave en el cliente.
H-03	P0	Respuestas predefinidas presentadas como respuestas de IA cuando fallaba la API.	Corregido	Se devuelve un error explícito.
H-04	P0	Credenciales de demostración en AuthScreen.kt, divididas en dos cadenas.	Corregido	Atajos solo en builds debug y sin contraseñas incluidas (commit 8dc030f).
H-05	P1	URL del webhook de Make en el código fuente (corregido: se inyecta al compilar).	Corregido	Se lee de local.properties o MAKE_WEBHOOK_URL; sigue extraíble del APK compilado (R-04).
H-06	P1	Faltaban gradlew, gradlew.bat y gradle-wrapper.jar.	Corregido	Wrapper restaurado.
H-07	P1	debug.keystore inexistente; la compilación debug fallaba en un equipo limpio.	Corregido	Se usa la firma debug estándar de Android.
H-08	P2	29 scripts fix_.*py y split_*.py generados por IA en la raíz.	Corregido	Archivados fuera del proyecto.
H-09	P2	README de AI Studio con instrucciones incorrectas.	Corregido	README reescrito.
H-10	P2	Proyecto sin Git y sin pipeline de CI.	Corregido	Repositorio y azure-pipelines.yml.
H-11	P2	AppContainer cae silenciosamente a repositorios simulados si Firebase falla.	Pendiente	Bug 46 en Azure Boards; ver R-02.
H-12	P2	Navegación duplicada (core/navigation y ui/navigation).	Corregido	Archivo sin referencias eliminado; compilación verificada.
H-13	P2	RetellAIRepository y N8nWebhookRepository son interfaces sin implementación.	Pendiente	Pendiente de decisión; ver R-03.
H-14	P2	Cualquier usuario autenticado podía leer solicitudes_atencion.	Corregido	El cliente lee solo las suyas; el administrador todas; reglas redesplegadas.
H-15	P2	El registro permite elegir el rol ADMINISTRADOR.	Pendiente	Bug 45 en Azure Boards; ver R-01.
H-16	P3	.env.example conserva la variable obsoleta GEMINI_API_KEY.	Corregido	.env.example ya no incluye la variable obsoleta.
H-17	P1	Si fallaba la lectura del rol, el usuario quedaba como ADMINISTRADOR.	Corregido	Mínimo privilegio: CLIENTE por defecto.
H-18	P1	El inicio de sesión usaba el rol elegido en pantalla, no el almacenado.	Corregido	El rol sale de la cuenta; si no coincide se rechaza el acceso.

ID	Prioridad	Hallazgo	Estado	Acción o evidencia
H-19	P1	El listener de solicitudes se abría antes del login y habría fallado con reglas autenticadas.	Corregido	Se usa AuthStateListener.
H-20	P2	SignUpUseCase, AuthRepository y AuthUser tenían ADMINISTRADOR como rol por defecto.	Corregido	El rol por defecto es CLIENTE (mínimo privilegio; commit 605a91d); ver R-16.
H-21	P1	El repositorio no contiene archivo LICENSE y la aplicación no implementa autorización ni política de tratamiento de datos personales.	Pendiente	Identificado en la revisión del SRS (allí figura como H-20; aquí se renumera H-21 porque AUDITORIA.md ya usa H-20). Pendiente de decisión del equipo.

Anexo D. Enlaces y recursos

Tabla 53. Enlaces del proyecto

Recurso	Dirección
Repositorio público del código	https://github.com/JuanXD2201/ServiX
Proyecto en Azure DevOps (privado)	https://dev.azure.com/Serivx/ServiX
Documentación del repositorio	Carpeta `docs/` (ARQUITECTURA, FIREBASE, CHAT_IA, PRUEBAS, AUDITORIA, AZURE_DEVOPS)
Documentos relacionados	DOC-02 Arquitectura, DOC-03 Plan de Pruebas, DOC-04 Informe de Pruebas, DOC-07 Firebase, DOC-08 IA, DOC-09 Azure DevOps